

面向任务的人类-Agent协作平台

架构设计基线是实现蓝图

L0 总体架构 · L1 模块拆解 · 实现证据 · ADR · L2 演进路线

版本：架构基线 v2.2.1

日期：2026 年 9 月

目录

文档状态

1. 如何阅读这份文档
2. 系统目标与非目标
3. 共享语言与系统对象
4. L0 总体架构
5. 统一模块设计方法
6. 设计哲学与思想来源
7. L1：Human & Device Experience 人类与设备交互层
8. L1：Task Collaboration & Control Plane 任务协作与控制平面
9. L1：Agent Runtime Agent 执行运行时
10. L1：接口与通信层 Communication Layer
11. L1：执行环境 Execution Environment
12. L1：持久化与事件基础 Persistence & Event Foundation
13. L1：人机协同闭环 Human-Agent Collaboration Loop
14. L1：System Agent 与治理平面
15. 端到端场景设计
16. 参考工程选择与源码依据
17. 安全、可靠性与治理要求
18. L2 实现路线
19. L2 设计评审清单
20. 当前待确认问题
21. 实现契约总览
22. 核心数据模型与关系
23. 状态机设计
24. 关键时序与跨模块流程
25. 代码仓库与包边界蓝图
26. API、事件和错误设计蓝图
27. 持久化、并发和恢复设计
28. 安全模型与权限矩阵
29. 可观测性、审计和解释
30. 测试与验收矩阵
31. 架构决策记录 ADR 索引
32. 从架构到代码的执行顺序
33. 完整性检查表

- 34. 优秀设计文档方法与本文件的对应关系
- 35. 需求追踪矩阵
- 36. 架构视图目录
- 37. 质量属性目标与验证方式
- 38. 部署拓扑与运行边界
- 39. 威胁模型与防护要求
- 40. 发布、运维与故障演练
- 41. 文档治理与变更流程
- 42. 参考实现边界与证据等级
- 43. L2 深化模板与下一轮输入
- 44. 第一版复杂度预算与最小公共模型
- 45. 目标代码结构与依赖方向
- 46. 关键逻辑伪代码
- 47. 开源源码定位与引用清单
- 48. ADR：第一版关键取舍
- 49. 版本化设计文档版式规范

文档状态

- 文档类型：系统架构设计基线
- 版本：v2.2.1
- 抽象范围：L0 总体架构、L1 模块架构、L2 实现设计蓝图
- 目标读者：产品架构师、系统架构师、后端与客户端工程师、Agent Runtime 工程师、平台治理工程师
- 适用工程：本工作区中的 codex、pi-mono、deepseek-harness、multica、ai-native
- 相关图件：/Users/ninja/work/projects/brain/架构图

这份文档描述一个面向 Task 的人类-Agent 协作平台。它把人类、多个 Agent、第三方工具和执行环境组织在同一个可恢复、可审计、可干预的任务闭环中。

本版本在 L0/L1 基础上继续下钻到 L2 设计蓝图。这里的 L2 是“实现应该如何分层、如何协作、如何验证”的设计约束，不是最终代码，也不替代具体 API、数据库 schema 和部署参数。任何代码落地都必须回指本文件中的一个模块、一个子模块和一个不变量。

本次 v2.2.1 修正了目录编号和有序列表排版，并延续 v2.2

的实现深度：增加文档摘要、复杂度预算、最小公共模型、代码结构图、关键伪代码、源码定位表和 ADR；同时保留详细状态机作为 L2 诊断视图，避免把运行时诊断能力误删出设计。

1. 如何阅读这份文档

1.1 三层抽象

层级	回答的问题	本版本的内容
L0	系统由哪些一级模块组成？模块边界是什么？	六个基础层、两个横向能力、总体任务主线
L1	一个模块内部有哪些子模块？各自负责什么？	子模块职责、输入输出、协作关系、闭环和边界
L2	如何把子模块落成代码和运行机制？	状态、端口、适配器、事件、目录、错误处理、测试和演进出口

L0、L1、L2 表示抽象层级，不表示网络层、部署层或调用顺序。一个 L1 模块可以在同一进程内实现，也可以跨进程实现；部署拆分必须服从模块职责，而不是反过来改变职责。

1.2 每个模块采用同一设计模板

后续阅读每个模块时，按以下顺序理解：

- 1. 模块目标：它为系统提供什么能力；
- 2. 责任边界：它明确不负责什么；
- 3. 子模块：每个子模块的职责和依赖；
- 4. 运行闭环：输入如何变成状态、动作、结果和证据；
- 5. 代码结构：领域、应用、端口、适配器和基础设施如何分层；
- 6. 参考工程：现有工程中可复用的代码模式和真实路径；
- 7. 设计哲学：为什么采用这种边界和约束；
- 8. L2 交付物：进入下一轮设计时必须明确的对象。

1.3 事实、建议和待设计内容

- 已实现依据：参考工程中可以定位到源码、测试或 README 的模式；
- 架构决策：本平台决定采用的边界和原则；
- 待设计内容：尚未在参考工程中完整实现，需要在 L2 继续讨论；
- 禁止臆造：不能把待设计内容写成某个参考工程已经提供的能力。

2. 系统目标与非目标

2.1 系统目标

1. 以 Task 作为人类和 Agent 的共同工作对象；
2. 让 Task Graph 表达任务拆解、依赖和推进关系；
3. 让 Agent Runtime 负责行动，让控制平面负责是否推进；
4. 让第三方 CLI、API、SDK 通过统一 Provider 和执行边界接入；
5. 让电脑端、移动端、CLI 和 Agent Client 看到一致的任务事实；
6. 让人工审批、补充信息、暂停、恢复和请求修改成为显式协作状态；
7. 让 Event、Artifact、Evidence、Audit 和 Checkpoint 支持回放、解释和恢复；
8. 通过 System Agent 主动观察、评估和介入 Agent 的关键节点；
9. 允许 Agent 能力逐步成长，但所有升级、降级和回滚都必须可解释。

2.2 第一版非目标

第一版不把复杂工作流引擎、开放式 Agent 社交网络、自动人格塑造、无边界记忆改写、大规模插件市场和完全无人值守的高风险执行作为前置条件。先实现一个小而完整的 Task 闭环，再扩展规划、治理和跨组织协作。

3. 共享语言与系统对象

3.1 Task

Task 是系统的首要业务对象，描述一个需要人类和 Agent 共同推进的目标。Task 持有目标、范围、参与者、当前状态、Task Graph 版本、关键 Artifact 引用和协作历史。

Task 不等于聊天会话，不等于 Agent Session，也不等于一次执行 Job。一个 Task 可以有多个 Session、多个执行 Job、多个人类参与者和多个 Agent。

3.2 Task Node 与 Task Graph

Task Node 是 Task Graph 中可独立推进的工作单元。Task Graph 负责表达依赖、并行、汇合、人工门和重新规划入口。第一版使用可解释的有限 DAG；循环只通过显式的重试、重新规划或恢复节点表达，不用隐式递归隐藏失败。

3.3 Actor、Agent 与 Session

- Actor：实际参与协作的主体，可以是 Human、Agent 或 System Agent；
- Agent：承担某类任务的运行角色；
- System Agent：观察和治理其他 Agent 的主动角色；
- Session：一次 Agent Runtime 的可恢复执行上下文；
- Client：Actor 进入系统的设备或客户端。

Actor 是身份和责任主体，Session 是运行时上下文，Client 是交互入口，三者不可混为一个对象。

3.4 Command、Event 与 Activity

- Command：参与者提出的意图，例如创建任务、审批、暂停和重试；
- Event：平台确认已经发生的事实，例如任务状态变更、工具执行结束和人工审批完成；
- Activity：面向用户的过程记录，可以由多个底层事件投影生成。

客户端只能提交 Command，不能直接写入共享事实状态。Event 由拥有状态的模块产生，并进入持久化和事件基础层。

3.5 Artifact、Evidence 与 Audit

- Artifact：任务产生或引用的文件、代码差异、报告、日志、截图和结构化输出；
- Evidence：支持某个判断、结果或状态变化的可追踪证据；
- Audit：记录谁在何时基于什么输入做了什么决定，以及决定造成的影响。

原始 Evidence 和已确认事实必须可追踪。派生 Memory、工作上下文和摘要可以重组、降权和版本化，但不能无痕覆盖原始证据。

3.6 Policy、Gate 与 Checkpoint

- Policy：判断某个动作是否允许的规则集合；
- Gate：任务推进前必须满足的人工或系统条件；
- Checkpoint：可恢复的状态和上下文快照。

Policy 决定允许性，Gate 决定推进条件，Checkpoint 决定恢复入口，三者不能被一个“万能状态字段”代替。

4. L0 总体架构



4.1 六个基础层

1. Human & Device Experience 人类与设备交互层：提供 Web、Desktop、Mobile、CLI 和 Agent Client 的统一入口；
2. Task Collaboration & Control Plane 任务协作与控制平面：组织 Task、Task Graph、Assignment、Handoff、Gate 和推进状态；
3. Agent Runtime Agent 执行运行时：驱动 Session、Agent Loop、模型、工具、扩展和运行时监督；
4. Communication Layer 接口与通信层：承载设备同步、Agent-to-Agent 协作和治理事件；
5. Execution Environment 执行环境：承载 Provider、CLI、API、SDK、Workspace、沙盒和资源边界；
6. Persistence & Event Foundation 持久化与事件基础：保存状态、Event、Activity、Artifact、Evidence、Checkpoint 和 Experience Ledger。

4.2 两个横向能力

- Human-Agent Collaboration Loop 人机协同闭环：把人类意图、Agent 提案、人工复核、受控执行和反馈连成循环；
 - System Agent & Governance Plane 系统 Agent 与治理平面：主动观察、评估、介入、校准和记录 Agent 的表现。
- 横向能力不是额外的“第七层”，而是跨越多个基础层的协作与治理能力。

4.3 总体任务主线

Create -> Understand -> Plan -> Assign -> Execute -> Review -> Complete

+----- Ask Human / Blocked / Retry / Replan / Rollback -----+

任务主线是业务语义，不是固定的微服务调用链。某些阶段可以合并，某些阶段可以重复，但每次重复必须有原因、版本和可恢复入口。

4.4 L0 边界不变量

- 控制平面决定“是否推进”，Runtime 决定“如何执行”；
- 通信层传递意图、事件和结果，不拥有业务决策权；
- 执行环境执行已授权能力，不直接改变 Task 事实；
- 持久化层保存事实和证据，不替代控制平面做决策；
- System Agent 可以提出治理动作，但不能绕过 Policy、Gate、证据来源和回滚；
- 人类和 Agent 使用同一个 Task 协作边界，但拥有不同的权限和责任。

5. 统一模块设计方法

5.1 领域层、应用层、端口层、适配器层

每个可独立演进的模块采用四类代码边界：

- Domain：状态、规则、不变量和领域事件；
- Application：用例编排、事务边界、幂等、重试和权限检查；
- Ports：模块对外需要的接口，例如 TaskStore、EventBus、ModelProvider 和 ExecutionProvider；
- Adapters/Infrastructure：HTTP、WebSocket、SQLite/Postgres、CLI、MCP、沙盒和第三方服务实现。

领域层不能依赖具体数据库、HTTP 框架或模型供应商。适配器可以变化，但不能改变领域规则。

5.2 事件和结果的共同格式原则

事件与结果至少需要能够回答：来源是谁、关联哪个

Task/Node/Session、发生时间、因果关系是什么、当前版本是什么、是否可重试、是否产生 Artifact、是否需要人工处理。

这些是设计要求，不在本版本规定具体 JSON 字段。字段、序列号、幂等键和协议版本属于 L2 实现规格。

5.3 失败处理的共同原则

每个异步操作都必须区分：等待、可重试失败、不可重试失败、取消、超时、人工接管和已回滚。错误不能只保存一段字符串；至少要保留阶段、来源、动作、上下文引用和恢复建议。

5.4 代码结构的共同原则

- 领域对象按模块归属，不建立跨模块共享的“万能 models”目录；
- 共享部分只放稳定协议、基础错误、ID 类型和事件信封；
- 接口方向由拥有能力的一方定义，调用方依赖端口；
- 同一进程调用和跨进程调用使用相同的业务语义，但传输细节隔离；
- 每个状态机拥有自己的 reducer/transition 入口，不允许多个模块随意写状态；
- 代码目录应能反映 L1 子模块，不用一个超大 orchestrator 隐藏所有逻辑。

6. 设计哲学与思想来源

6.1 控制论：用反馈代替一次性完美规划

Wiener 和 Ashby 的控制论强调观察、比较、纠偏和再次行动。平台因此采用 Observe -> Evaluate -> Intervene -> Measure 的闭环。Task Graph 不是一次生成后永不改变的计划，而是带版本、证据和反馈的可修订计划。

对应工程含义：保留过程事件，记录偏差来源，允许重试和重新规划；不要把模型第一次输出当成最终真相。

6.2 负反馈与“暗面”能力

系统需要质疑、证据挑战、风险识别和发散抑制等负反馈能力。它们不是恶意 Agent，也不是对用户的永久人格判断，而是用于校准系统的对抗性观察器。

对应工程含义：暗面角色只能产生带来源和作用域的建议；建议必须经过 Policy 或人工门才能影响事实；其介入必须可审计和可回滚。

6.3 最小权限与能力安全

能力安全模型要求对象只能持有完成任务所需的最小能力。Provider、Tool、Workspace、凭证和网络访问都应通过显式授权获得。

对应工程含义：System Agent 不拥有万能权限；第三方 CLI 不能因为被注册就默认获得文件、网络 and 凭证访问；高风险动作必须经过 Policy Gate。

6.4 认识论与证据来源

Agent 的判断不是事实本身。平台把原始输入、执行观察、人工决定、派生摘要和治理建议分开保存，使“知道什么”和“推断什么”可以被追溯。

对应工程含义：Memory 压缩可以整理上下文，但不能无痕删除原始 Evidence；任何冲突都应保留来源并允许重新评估。

6.5 社会技术系统与人类能动性

人类不是流程中的最后一个审批按钮，而是可以提出目标、改变约束、补充知识、暂停任务和接管执行的协作主体。Agent 的自动化程度应随着证据质量、任务风险和历史表现动态调整。

对应工程含义：HITL 需要支持补充信息、请求修改、局部批准、暂停、恢复和接管，而不只是 Allow/Reject 两个按钮。

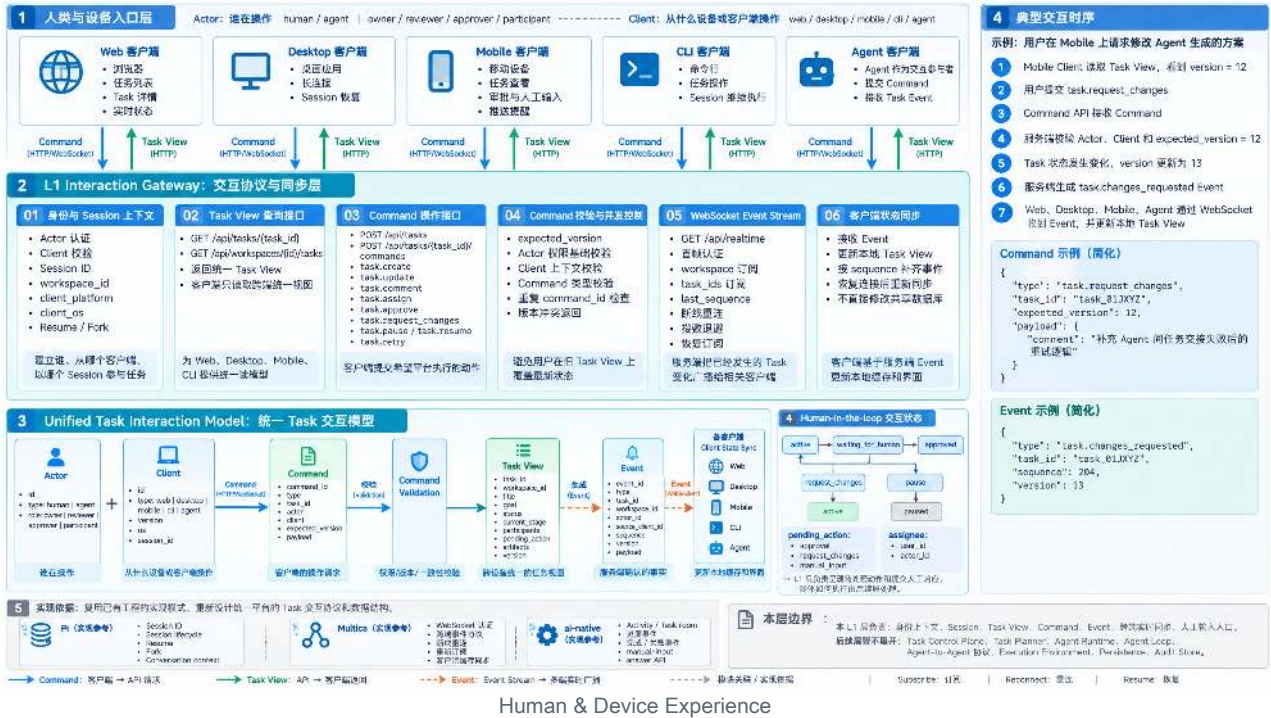
6.6 可逆性优先

在不确定的自动化系统中，可逆比一次性正确更重要。高影响动作需要 Checkpoint、Artifact、Audit 和 Rollback 入口。

对应工程含义：执行器在修改 Workspace 前创建恢复点；治理动作记录旧版本和新版本；升级和降级都可以撤回。

7. L1 : Human & Device Experience 人类与设备交互层

Agent 协作平台 L1 架构：人类与设备交互层 (Human & Device Experience)



7.1 模块目标与边界

该模块让 Human、Agent 和 System Agent 通过 Web、Desktop、Mobile、CLI 或 Agent Client 进入同一个 Task。它负责展示、输入、同步和人工介入体验，不負責任务图规划、模型推理、工具执行和事实存储。

7.2 子模块总表

子模块	核心职责	主要输入	主要输出
Client Applications	提供不同设备的交互入口	Task View、Event、通知	Command、输入、审批
Participant Context	关联 Actor、Client、Role、Session	登录身份、设备信息	权限上下文、关联 ID
Task Interaction Gateway	统一读视图和写意图	Command、Query	Task View、确认结果
Device Sync Channel	跨端订阅、补发、重连	Event、订阅请求	实时更新、同步状态
HITL Experience	呈现待办和收集人工决定	Human Gate、Proposal	Approval、Change Request、Input

7.3 子模块设计

7.3.1 Client Applications

Web、Desktop、Mobile、CLI 和 Agent Client 共享 Task 语义，但不共享全部 UI。客户端只处理视图状态、输入校验、乐观展示和本地缓存，不直接修改服务端事实。

客户端代码建议按平台壳和共享领域视图分离：

apps/web 页面、路由、服务端渲染适配
apps/desktop 桌面壳、窗口、通知、设备能力

apps/mobile 移动页面、离线缓存、推送入口
 apps/cli 命令解析、终端渲染、脚本出口
 packages/task-view Task View、事件归并、权限化展示
 packages/client-core Query、Command、同步和本地状态端口

参考 Multica 的 Web/Desktop/Mobile 共用工作区思路，但不把其 Issue 数据模型直接作为本平台 Task 模型。

7.3.2 Participant Context

Participant Context 将 Actor、Client、Workspace、Role、Task 和 Session 关联起来。它需要支持一个 Actor 多设备、一个 Task 多参与者、一个 Agent 多 Session。

关键设计：上下文是一次请求或连接的关联信息，不是永久写入 Task 的人格标签；用户能力校准只用于解释深度、确认强度和自动化程度。

7.3.3 Task Interaction Gateway

Gateway 将 Query 转换为 Task View，将 Command 转交控制平面，并把控制平面确认的 Event 投影为客户端可读结果。它不在网关内重新实现 Task Graph 或权限规则。

建议的用例入口：CreateTask、GetTaskView、SubmitCommand、SubscribeTask、AcknowledgeEvent、RespondTo HumanGate。具体接口协议进入 L2。

7.3.4 Device Sync Channel

同步通道支持订阅、心跳、断线检测、序列恢复、去重、补发和背压。客户端恢复时先取得一个版本化 Task View，再从指定序列继续接收 Event，避免只依赖实时消息。

参考 Multica 的 WebSocket 订阅和 Mobile 查询缓存；参考 Codex App Server 的连接清理和实时事件处理。可靠性逻辑由通信层提供，客户端只消费统一语义。

7.3.5 HITL Experience

人工介入页面应呈现：当前任务目标、Agent

提案、影响范围、Evidence、风险、可选动作、需要补充的信息和撤销路径。Approval、Request Changes、Provide Input、Pause、Resume、Take Over 是不同意图，不压缩成布尔值。

7.4 运行闭环

```
Query -> Task View -> Human/Agent Command -> Control Plane
      ^               |
      +---- Event / Notification / Sync -----+
```

7.5 代码结构与实现约束

- 客户端查询使用统一 Query 端口，服务端状态通过缓存或 React Query 类机制管理；
- 客户端临时状态与服务端事实分离；
- WebSocket 事件只更新服务端状态缓存，不让多个本地 Store 互相写同一事实；
- UI 组件只负责展示和触发意图，权限和状态迁移由服务端拥有；
- Agent Client 必须走与 Human 相同的 Gateway，不设置绕过入口。

7.6 参考工程与代码依据

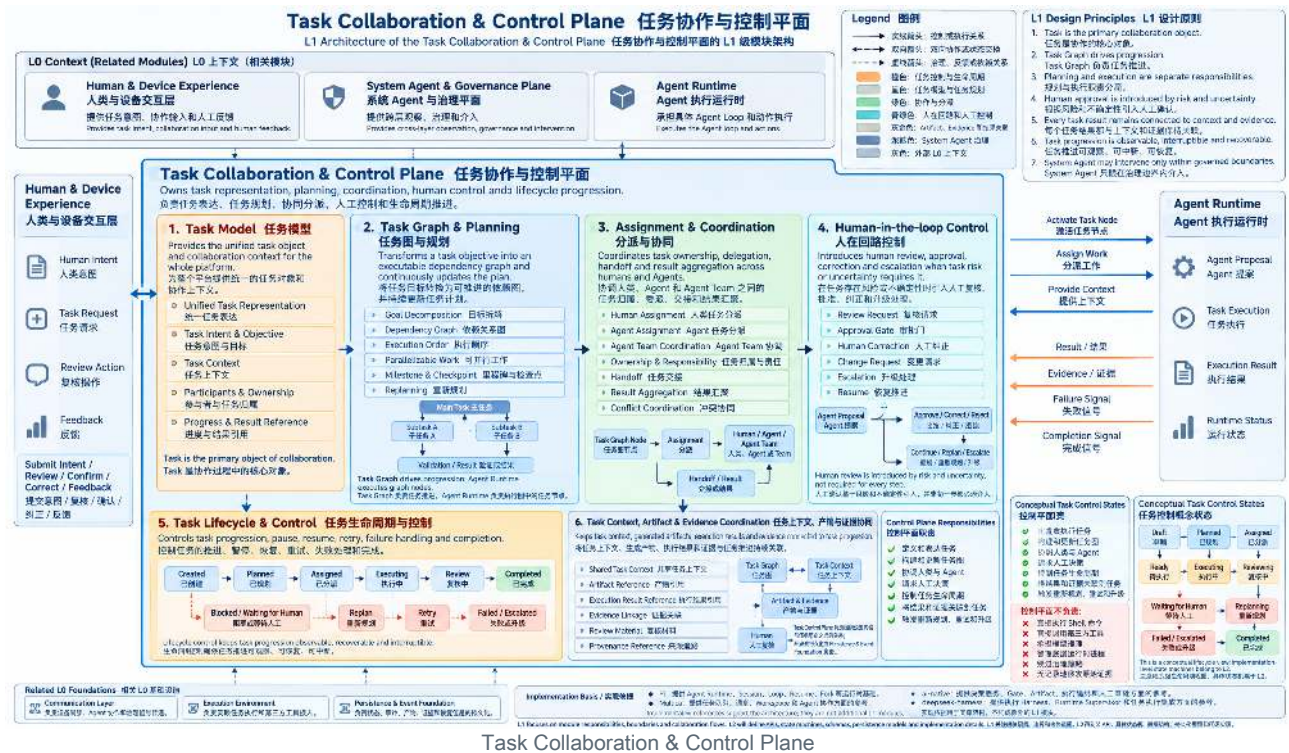
- Multica：apps/mobile/data/、apps/mobile/lib/use-ws-subscriptions.ts、server/internal/realtime/；
- Codex：codex-rs/app-server/src/transport.rs、realtime_event_handling.rs、connection_cleanup.rs；
- ai-native：前端 Task/Activity 展示与 Executor 状态回传；
- Pi：Session 恢复和持续交互的客户端语义。

7.7 L2 交付物

Actor/Client/Task View/Command/Event 的字段；客户端状态归属；订阅和重连协议；人工门展示模型；移动端离线策略；权限化视图；源码目录与组件测试边界。

8. L1 : Task Collaboration & Control Plane

任务协作与控制平面



8.1 模块目标与边界

该模块是 Task 的业务控制中心。它组织 Task、Task Node、Task Graph、Assignment、Handoff、Human Gate 和 Task State，向 Runtime 发出执行意图，接收结果后决定继续、重试、暂停、重新规划或完成。

它不负责 UI 渲染、模型推理、工具执行和底层存储实现。

8.2 子模块总表

子模块	核心职责	关键不变量
Task Model	维护任务目标、参与者和版本	Task 是协作主对象
Task Graph	表达拆解、依赖、并行和汇合	依赖可解释、版本可追踪
Assignment	分配、领取、租约和回收	一个节点的责任主体明确
Handoff	人和 Agent 间交接	交接有输入输出引用
Human Gate	人工审核、补充、接管	通过条件显式记录
Task State	维护业务状态	只有拥有者能迁移状态
Planner/Orchestrator	规划和推进用例	不把执行细节塞进控制器

8.3 子模块设计

8.3.1 Task Model

Task Model 管理目标、约束、优先级、参与者、Graph 版本、结果和 Artifact 引用。它只包含协作事实，不保存完整 LLM 上下文。

建议代码结构：

```
task-control/domain/task/
  task.ts           Task 聚合和不变量
  task-node.ts      Node 与责任关系
  task-state.ts     状态枚举和迁移规则
  task-events.ts    Task 领域事件
task-control/application/
  create-task.ts
  update-task.ts
  transition-task.ts
  request-human-action.ts
```

8.3.2 Task Graph

Task Graph 分为三步：生成候选图、校验图、发布图版本。Planner 只能提交候选图；Control Plane 校验节点唯一性、依赖闭合、无非法环、人工门位置和能力要求后，才能发布版本。

第一版只支持有限 DAG、显式 Retry/Repair 节点和可追踪 Replan。复杂搜索、动态资源优化和跨 Task 图组合留到后续。

8.3.3 Assignment、Handoff 与协作

Assignment 说明谁负责、需要什么能力、租约何时开始和如何回收。Handoff 说明交接原因、输入引用、当前状态、待解决问题和下一责任主体。

Agent-to-Agent 协作必须通过 Task Node、Assignment 和 Artifact/Evidence 引用发生；不能用任意全局消息取代责任边界。

8.3.4 Human Gate

Human Gate

包含待审议对象、风险说明、Evidence、允许动作、截止时间和默认恢复策略。通过后由控制平面产生状态迁移 Event；拒绝或请求修改进入明确的 Replan/Repair 分支。

8.3.5 Task State

建议状态集合：Draft、Planned、Ready、Running、WaitingForHuman、Blocked、Review、Completed、Failed、Cancelled、RolledBack。状态集合不是最终 API；每个状态必须定义进入条件、离开动作、可重试性和恢复点。

8.4 运行闭环

```
Create -> Plan -> Validate -> Assign -> Dispatch
  ^               |
  |               v
Replan <- Review <- Result/Evidence <- Runtime
```

8.5 代码结构与实现约束

建议采用领域聚合 + 应用用例 + 端口适配器：

```
task-control/
  domain/task/      Task、Node、State、Events
  domain/graph/     Graph、Dependency、PlannerProposal
  domain/coordination/ Assignment、Lease、Handoff
  domain/gates/     HumanGate、Review、Escalation
  application/      use case handlers
```

ports/	TaskStore、RuntimePort、EventPublisher
adapters/	HTTP、DB、Runtime client、Event projection
tests/	transition、graph、recovery、integration

控制器只编排用例，不直接调用模型或 Shell。状态迁移使用单一入口和显式原因，失败恢复通过应用用例处理。

8.6 参考工程与代码依据

- Multica：server/internal/dispatch/、server/internal/daemon/、server/internal/events/、server/internal/service/；
- ai-native：backend/domain/gates/、backend/stages/、backend/service_layer/、backend/execution-jobs/；
- Codex：Thread 状态、App Server 请求处理和持久化边界；
- Pi：Session 与 Runtime 的恢复和分支语义。

8.7 L2 交付物

Task/Node/Graph/Assignment/Handoff/Gate 的字段和版本规则；状态迁移表；Planner Proposal 校验；Runtime Intent/Report 协议；租约、幂等、并发控制；失败和回滚策略；控制平面测试矩阵。



Runtime 不拥有 Task 的业务状态，不直接决定人工门是否通过，不绕过 Execution Environment 使用第三方工具。

9.2 子模块总表

子模块	核心职责	参考实现方向
Session Runtime	启停、恢复、取消和分支	Pi Session
Context & Memory	上下文构建、压缩和召回	Pi harness/session、ai-native context
Pi Agent Loop	状态化循环、工具调用、事件流	Pi agent-loop.ts
JEV Acceleration	场景级快速选择、编排和异步整理	本平台新增接入点
Model Provider	多模型流式调用和能力声明	Pi AI provider
Tool Provider	工具发现、授权和调用端口	Pi tools、dsh capability seam
Extension Runner	命令、扩展和事件订阅	Pi extensions、dsh plugins
Runtime Supervisor	超时、心跳、并发和异常收敛	Multica daemon、Codex server
Result Writer	输出、Artifact、Evidence 和报告	ai-native execution-result

9.3 子模块设计

9.3.1 Session Runtime

Session 是可恢复的运行时上下文，至少关联 Task、Node、Actor、Runtime、Model 配置和当前 Checkpoint。Session 的生命周期由 start、turn、tool execution、pause、resume、cancel、end 组成。

Pi 的 Agent、agent-loop.ts、harness session 和 JSONL 事件流提供了直接的实现参考。平台需要把 Session 与 Task 解耦：Task 是业务事实，Session 是可替换的执行上下文。

9.3.2 Context & Memory

Context Builder 根据 Task 目标、当前 Node、Session 事件、工具结果、Artifact 摘要和治理建议构造模型上下文。Memory Organizer 可以异步执行裁剪、摘要、降权、去重和召回建议。

必须区分：原始 Session 事件、派生摘要、工作上下文和治理建议。异步整理不能覆盖原始事件，也不能在没有版本和来源的情况下改变模型可见内容。

9.3.3 Pi Agent Loop

Loop 的基本周期是：读取当前状态、调用模型、处理流式输出、校验 Tool Call、执行工具、写入 Tool Result、决定是否继续。工具可以串行或并行，但结果顺序、取消和错误语义必须明确。

代码组织建议直接模仿 Pi：

```
runtime/agent/
  agent-loop.ts
  agent.ts
  stream-fn.ts
  messages.ts
  tools.ts
  events.ts
  state.ts
runtime/harness/
  context.ts
  session.ts
  compaction.ts
  telemetry.ts
```

9.3.4 JEV Acceleration

JEV 位于 Agent Loop 内部，不是新的顶层架构层。它适合处理高频、低延迟、可验证的局部动作：工具候选排序、动作模板选择、上下文异步整理、结果初步校验和场景级 Loop 加速。

JEV 不能绕过控制平面、Policy Gate、人工门和 Evidence 写入。它输出的是候选动作、加速结果或验证信号，最终业务状态仍由拥有者确认。

9.3.5 Provider 与 Extension

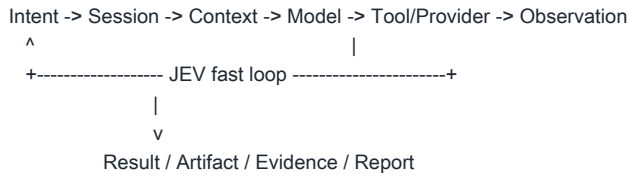
Model Provider 提供统一流式接口；Tool Provider 提供能力声明、参数校验、权限请求、执行和结果转换；Extension 通过生命周期钩子增加行为，不直接修改核心状态。

DeepSeek Harness 的 Service Definition / Provider / Consumer 分层适合约束能力扩展；Pi 的扩展和命令机制适合交互层扩展。两者要通过明确的端口连接，不能把插件、Provider 和 Runtime 混成一个目录。

9.3.6 Runtime Supervisor

Supervisor 处理取消、超时、并发上限、心跳、重试、崩溃恢复、资源释放和结果收敛。它报告 Runtime 状态，不直接把 Runtime 状态当成 Task 状态；Task 状态由控制平面映射。

9.4 运行闭环



9.5 代码结构与实现约束

建议以 TypeScript 状态机和端口为中心，遵循 Pi 的事件流和显式状态风格：

```

agent-runtime/
domain/session/    Session、Turn、RuntimeState
domain/context/    ContextItem、MemoryView、Compaction
application/       run-session、resume、cancel、compact
ports/model/       ModelProvider
ports/tools/       ToolProvider、ToolRegistry
ports/execution/   ExecutionPort
adapters/pi/       Pi loop/session/tool adapter
adapters/jev/      JEV adapter
adapters/providers/ OpenAI/Anthropic/other providers
supervision/       timeout、retry、heartbeat、resource
reporting/         runtime event、result、evidence
  
```

禁止在 agent-loop 中直接写入数据库或调用某个具体 CLI；禁止用一个巨型 Agent 类承载模型、上下文、工具、权限和持久化。

9.6 参考工程与代码依据

- Pi : packages/agent/src/agent-loop.ts、agent.ts、harness/session/、harness/tools/、harness/compaction/；
- DeepSeek Harness : packages/core/、packages/llm/、packages/tool/、packages/interaction/，以及 docs/architecture.md；
- Multica : server/internal/daemon/、runtimeapps/、dispatch/；
- ai-native : executor/backend/runtime/、executor/adapters/、executor/learning/。

9.7 L2 交付物

Session 状态机；Agent Event Map；Context 可见性和大小上限；Tool Call 并行/串行规则；Provider 端口；JEV 接入协议；取消、重试和超时语义；Runtime Report；性能指标和 Loop 测试。

10. L1：接口与通信层 Communication Layer

接口与通信层 Communication Layer 的 L1 模块架构



10.1 模块目标与边界

通信层只负责连接、关联、消息传递和恢复。第一版重点是设备同步通道和 Agent-to-Agent 协作通道。通信层不做 Task 决策、不执行工具、不持有 Agent 的私有推理状态。

10.2 子模块设计

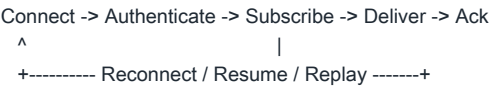
子模块	设计内容
Connection Management	认证、心跳、租约、断线和连接清理
Session/Correlation	关联 Actor、Client、Task、Node、Session、Request
Device Sync	订阅、快照、序列恢复、补发、去重和背压
Agent Collaboration	Handoff、能力询问、结果引用和协作状态
Governance Channel	Policy、Gate、Audit 和介入事件
Reliability	幂等、顺序、超时、重试、死信和降级

10.3 Agent-to-Agent 通信设计

Agent 间通信不是开放聊天，而是以 Task Node 为边界的协作协议。发送方必须说明目标 Node、责任关系、输入引用、期望输出、超时和失败处理；接收方返回结果引用、证据和状态，而不是只返回自然语言。

允许的消息类型包括：能力询问、任务交接、执行请求、结果报告、证据请求、阻塞通知和人工升级。禁止通过 Agent 消息直接修改其他 Agent 的权限、Task 状态或原始 Evidence。

10.4 运行闭环



10.5 代码结构与参考风格

communication/	
domain/connection/	Connection、Lease、Presence
domain/message/	Envelope、Correlation、Sequence
application/	subscribe、publish、resume、handoff
ports/transport/	WebSocket、HTTP、RPC、Queue
adapters/device/	Web、Desktop、Mobile、CLI
adapters/agent/	Agent collaboration adapter
reliability/	dedup、retry、backpressure
audit/	delivery and correlation records

参考 Multica 的 realtime/daemonws、Codex App Server 的 transport/RPC/connection cleanup、ai-native 的 job/result 与 heartbeat。DeepSeek Harness 的 Cordis 事件组合只能作为进程内扩展模式，不能替代跨设备协议。

10.6 L2 交付物

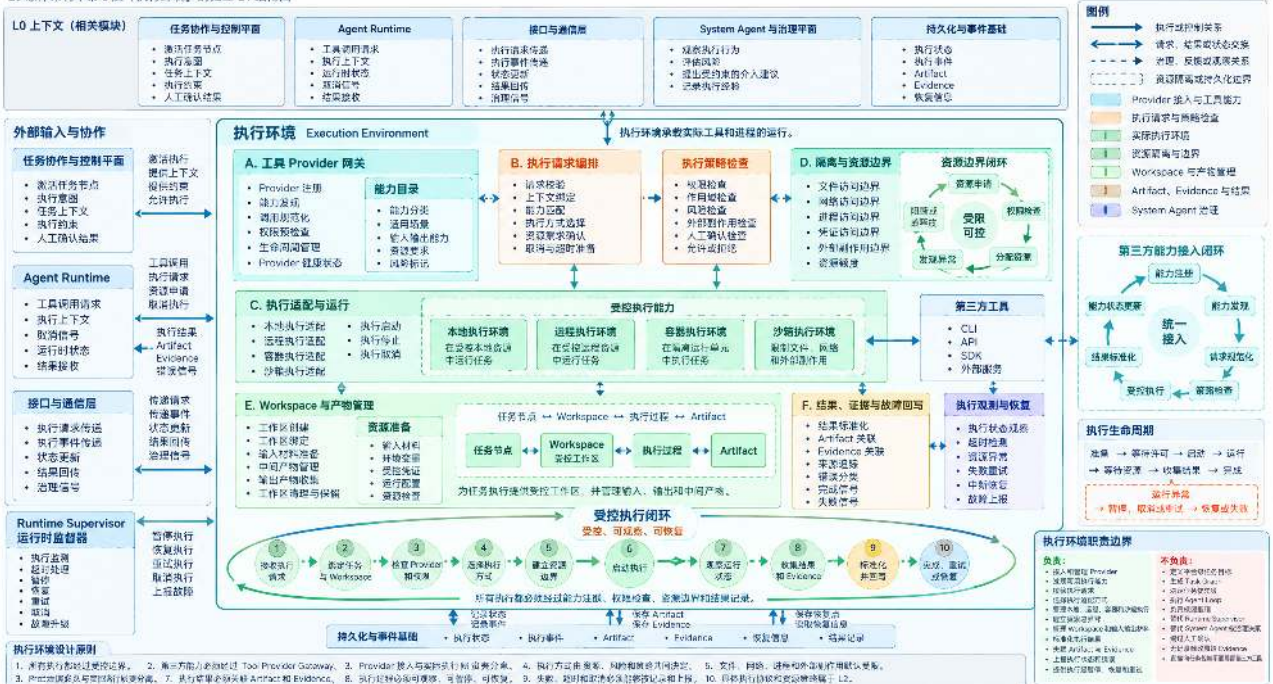
消息信封、序列号、幂等键、订阅恢复、Agent 协作消息、连接状态机、权限校验、死信和可观测性字段。

11. L1：执行环境 Execution Environment

执行环境 Execution Environment 的 L1 模块架构

L0 总体架构中第 5 层「执行环境」的独立 L1 细化图

为 Agent 和第三方能力提供受控、可观测、可恢复的实际执行能力。



Execution Environment

11.1 模块目标与边界

执行环境承接已授权的执行请求，把 Provider 能力落到本地进程、第三方 CLI、API、SDK、Workspace 或远程节点。它负责隔离和观察，不负责规划 Task，也不直接修改控制平面状态。

11.2 子模块设计

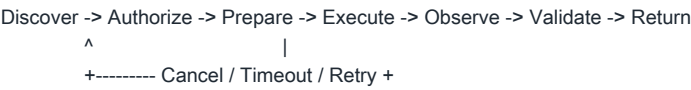
子模块	设计内容
Provider Gateway	注册、发现、版本、健康、输入输出和适配
Capability Catalog	工具能力、资源需求、风险和限制
Execution Orchestrator	Job 排队、租约、取消、并发和结果收敛
Policy Gate	文件、进程、网络、凭证和高风险动作授权
Execution Adapter	Shell、File、Browser、HTTP、CLI、IDE、MCP
Isolation Boundary	Workspace、容器/沙盒、资源和网络边界
Artifact Workspace	文件、差异、日志、输出和中间产物
Result/Evidence Return	退出原因、结果、证据和恢复信息

11.3 Provider 设计

Provider 是外部能力适配器，不等于插件，也不等于 Agent。Provider 声明能力并实现执行端口；Plugin 可以注册 Provider 或扩展 Runtime；Policy Gate 决定当前 Task 是否能调用 Provider。

第三方 CLI 的执行步骤固定为：发现能力、解析参数、检查策略、准备 Workspace、创建 Job、执行并观察、校验结果、保存 Artifact、返回 Evidence。任何一步失败都要产生结构化失败原因。

11.4 运行闭环



11.5 代码结构与参考风格

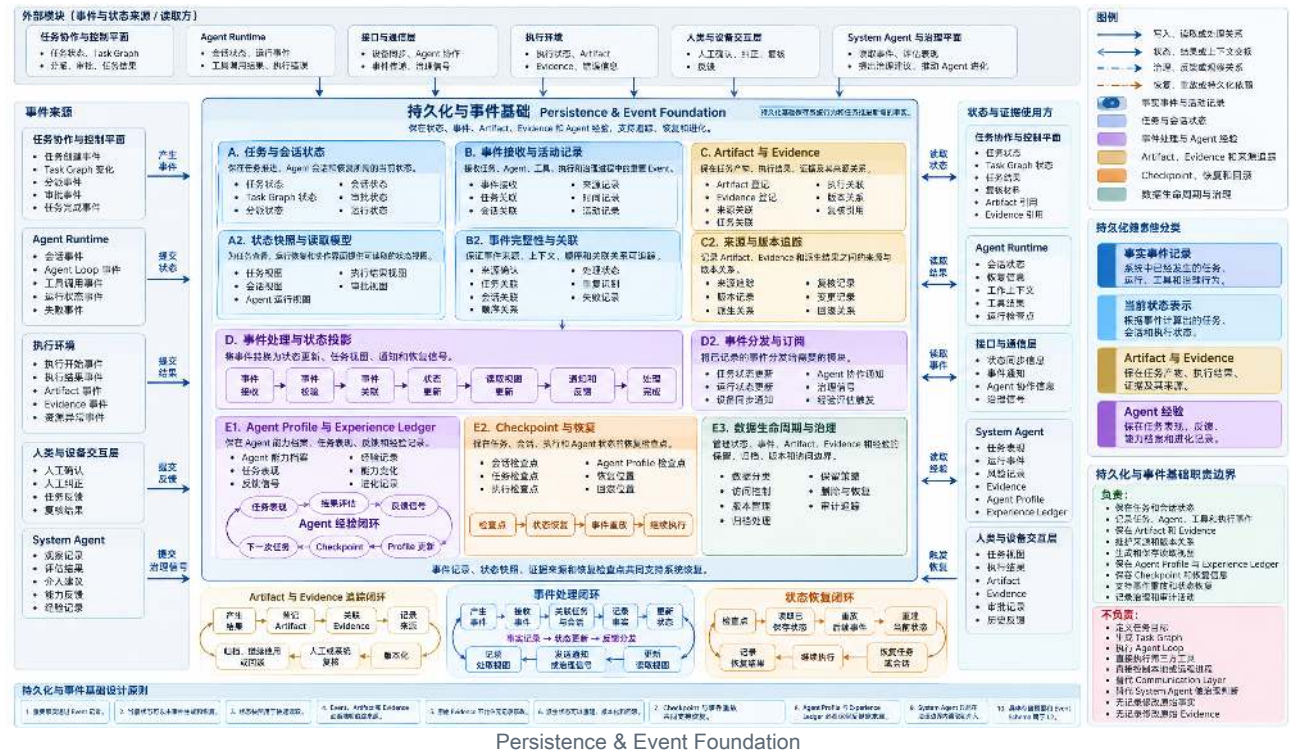
execution/	
domain/provider/	Provider、Capability、Risk
domain/job/	Job、ExecutionState、Result
application/	dispatch、cancel、retry、recover
ports/policy/	PolicyGate、CredentialPort
ports/runtime/	Local、Remote、Container
adapters/cli/	CLI adapter
adapters/mcp/	MCP adapter
adapters/http/	API/SDK adapter
sandbox/	filesystem、network、process boundary
workspace/	prepare、snapshot、artifact
observation/	logs、metrics、exit reason

ai-native Executor 的注册、能力选择、Workspace 准备、IDE Adapter 和 execution-result/v1 是直接参考；Multica 的 daemon/runtime 和失败重试提供本地运行模式；Pi 提供工具参数校验和执行模式；Codex 的 execpolicy、sandboxing 和 rollout-trace 提供权限、隔离和证据边界。

11.6 L2 交付物

Provider 能力描述；Job 生命周期；Policy 规则；沙盒边界；凭证注入；Workspace 快照；CLI/API/MCP Adapter；取消和超时；结果校验；Artifact 和 Evidence 回写。

12. L1：持久化与事件基础 Persistence & Event Foundation



Persistence & Event Foundation

12.1 模块目标与边界

持久化和事件基础保存系统事实、过程和证据，并将事件投影成 Task View、Timeline、通知、治理输入和恢复状态。它不负责规划、推理或授权决策。

12.2 子模块设计

子模块	设计内容
State Store	Task、Node、Assignment、Session 和连接状态
Event Store	事件信封、顺序、来源、因果和幂等
Projection	Task View、Activity、Timeline、Notification
Artifact Store	文件、差异、日志、报告和引用
Evidence Store	观察、验证、来源和结论支持关系
Checkpoint	Session、Graph、Workspace 和恢复点
Audit	决策、权限、介入、人工动作和回滚
Experience Ledger	Agent 表现、反馈、能力缺口和升级依据

12.3 事件与投影设计

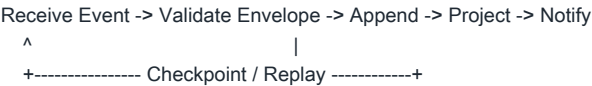
事件由拥有状态的模块产生，持久化层负责保存和投影。投影可以重建，原始事件不可被投影结果覆盖。Activity 是面向用户的派生视图，不是唯一事实来源。

第一版可以使用关系数据库 + Event/Activity 表 + Artifact
文件存储，不强制引入完整事件溯源平台。只有需要回放、审计或恢复的事实才进入不可变事件流。

12.4 记忆与证据边界

Memory Organizer 可以对工作上下文进行摘要、去重、降权、压缩和召回，但必须保留来源事件、版本、生成器和撤销关系。任何“虚假记忆”或对抗性建议都只能作为标注的派生内容存在，不能覆盖事实。

12.5 运行闭环



12.6 代码结构与参考风格

persistence/	
domain/event/	Event、Envelope、Causality
domain/artifact/	Artifact、Evidence、Provenance
application/	append、project、replay、recover
ports/store/	EventStore、StateStore、ArtifactStore
adapters/sql/	SQLite/Postgres adapter
adapters/blob/	local/object storage adapter
projections/	task-view、activity、timeline、audit
retention/	lifecycle、redaction、versioning

Pi 的 Session/JSONL、分支和压缩边界；Codex 的 Thread Store、Rollout Trace、Realtime History；Multica 的 Activity、运行日志和审核时间线；ai-native 的 Pipeline/Execution 快照都可以作为实现参考。

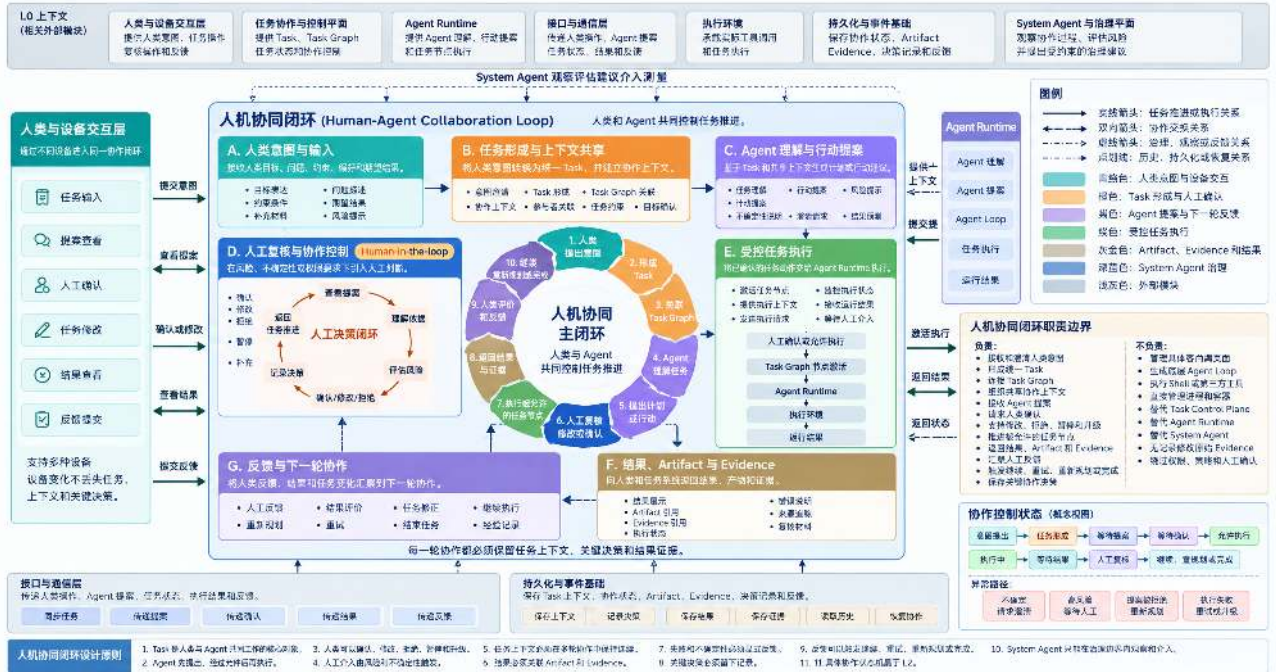
12.7 L2 交付物

事件信封和版本；Event Store、State Store、Artifact Store
端口；投影重建；证据来源；Checkpoint；Audit；保留和删除策略；一致性和恢复测试。

13. L1：人机协同闭环 Human-Agent Collaboration Loop

人机协同闭环 Human-Agent Collaboration Loop 的 L1 模块架构

系统 Task 组织人与 Agent 的意图、提案、确认、执行、复核和反馈。



Human-Agent Collaboration Loop

13.1 目标

人机协同闭环把人类的意图、Agent 的理解、执行提案、人工复核、受控执行、结果证据和反馈组织为多轮循环。它不是审批页面，也不是另一个 Runtime。

13.2 协同状态机

建议状态：IntentCaptured、TaskFormed、ProposalReady、AwaitingReview、Approved、ChangesRequested、Executing、EvidenceReady、Completed、Paused、Escalated、RolledBack。

状态迁移必须有触发者、输入引用、Evidence、下一责任主体和恢复策略。人类可以在 AwaitingReview、Executing 和 EvidenceReady 阶段介入；Agent 可以请求输入、报告阻塞和提交提案；System Agent 可以提出风险或暂停建议。

13.3 代码结构

```
collaboration/
domain/intent/      HumanIntent, AgentProposal
domain/review/      Review, Approval, ChangeRequest
domain/feedback/    Feedback, Escalation, TakeOver
application/        propose, review, execute, resume
ports/control/       TaskControlPort, RuntimePort
ports/interaction/   HumanInteractionPort
projections/         collaboration-view, inbox, timeline
```

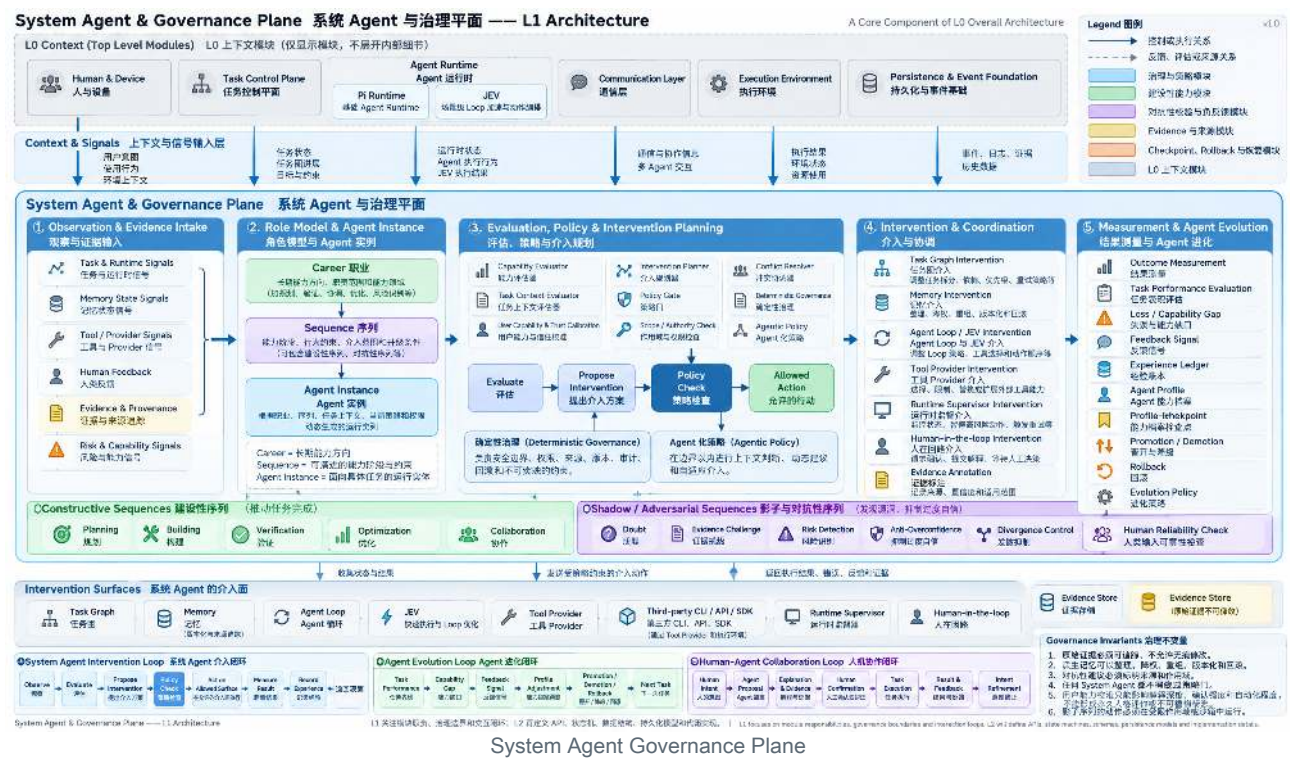
13.4 设计哲学

人类介入不是自动化失败后的补丁，而是系统中的合法决策节点。自动化程度应由风险、证据、历史表现和用户偏好共同决定；低风险重复任务可以减少确认，高影响和不确定任务必须提高确认强度。

13.5 L2 交付物

协同状态机；Proposal/Review/Feedback
对象；人工门协议；接管和回滚；多设备提醒；权限化解释；协同闭环测试。

14. L1 : System Agent 与治理平面



14.1 角色模型 : Career、Sequence、Instance

系统不预设固定数量的系统角色。角色以三层抽象组织：

- Career：长期能力方向，例如规划、验证、协作、风险控制；
- Sequence：阶段性能力要求、行为约束、介入权限和升级条件；
- Agent Instance：绑定到具体 Task 或治理范围的运行实例。

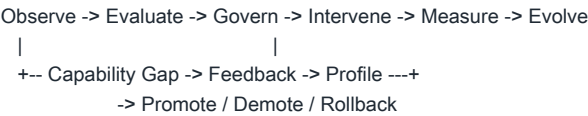
Sequence

是对能力的约束和训练路径，不是永久人格。升级需要任务表现、证据和反馈；降级或回滚同样是合法状态。

14.2 关键介入点

介入点	System Agent 观察什么	允许产生的动作
Memory/Context	冗余、冲突、过时和召回质量	摘要、降权、冲突标注、召回建议
Task Graph	拆解质量、依赖遗漏和风险	质疑、补充、请求重规划
Agent Loop	停滞、循环、过度发散和工具选择	限速、暂停建议、替代工具建议
Tool Provider	能力匹配、权限和第三方风险	拒绝建议、升级人工、要求证据
Runtime Supervisor	超时、重复、资源和失败模式	取消、重试、切换执行节点
Human-in-the-loop	用户意图和确认强度	调整解释和确认策略
Evidence	结论支持、来源冲突和缺证	要求补证、标记风险、触发回滚

14.3 进化闭环



System Agent 的动作必须记录原因、作用域、来源、版本、影响和撤销路径。治理建议和业务事实分离，建议被采纳前不改变 Task 状态。

14.4 代码结构

system-agents/	
domain/career/	Career、Sequence、PromotionRule
domain/profile/	AgentProfile、Capability、Experience
domain/evaluation/	Evaluation、EvidenceScore、Gap
domain/intervention/	Intervention、Scope、Reversal
application/	observe、evaluate、intervene、evolve
ports/	TaskPort、MemoryPort、PolicyPort、AuditPort
adapters/	memory、graph、runtime、provider、human
policy/	positive、adversarial、safety constraints
ledger/	experience、feedback、promotion history

系统 Agent 可以通过插件或 Provider 接入，但核心治理动作必须经过统一 Application 和 Policy Port。不能让每个 System Agent 自己持有一套事实数据库。

14.5 正向与暗面能力

正向能力负责规划、构建、验证、优化和协作；暗面能力负责质疑、证据挑战、风险暴露、过度自信抑制和发散控制。暗面角色是负反馈观察器，不是恶意人格，也不能修改原始事实。

14.6 L2 交付物

Career/Sequence/Instance 模型；评价指标；证据评分；介入范围；升级/降级规则；策略版本；治理审计；回滚；对抗性评估和误报处理。

15. 端到端场景设计

15.1 一个代码修改任务

1. Human Client 提交自然语言意图；
2. Gateway 创建 Task，控制平面生成初始 Task Graph；
3. Planner/Validator 检查依赖和人工门；
4. Assignment 选择具备代码能力的 Agent 和 Runtime；
5. Runtime 创建 Session，Pi 风格 Loop 读取 Context；
6. Tool Provider 请求 Execution Environment 创建受控 Workspace；
7. Agent 产生修改、测试结果、Artifact 和 Evidence；
8. System Agent 检查风险、证据和回归范围；
9. Human 在 Review Gate 复核；
10. Control Plane 完成、重试、请求修改或回滚；
11. Persistence 层保存 Event、Activity、Artifact、Audit 和 Checkpoint；
12. Device Sync 将结果同步到 Web、Desktop、Mobile 和 CLI。

15.2 一个多 Agent 协作任务

主 Agent 创建多个 Task Node，分别交给分析 Agent、实现 Agent 和验证 Agent。每个 Agent 只通过 Node、Assignment、Artifact 和 Evidence 交接；主 Agent 不能因为拥有编排责任就绕过 Provider、Policy 或 Human Gate。

15.3 记忆整理任务

JEV 或 System Agent 在后台观察 Context，生成压缩候选、冗余标记和冲突报告。候选内容写入派生 Memory 版本，原始事件保留；高影响冲突进入人工或治理复核。用户主交互不被慢速整理阻塞。

16. 参考工程选择与源码依据

16.1 选择矩阵

模块	主要参考	复用的模式	不直接照搬
Agent Runtime	Pi	Agent Loop、Session、事件流、工具生命周期、Provider	Pi 的产品边界和领域模型
扩展组合	DeepSeek Harness	Cordis 生命周期、Service Definition/Provider/Consumer、插件组合	插件直接改变核心事实
任务协作	Multica	Task/Issue、Agent 作为队友、Runtime、Activity、审核、重试	Issue/Squad 完整数据模型
决策与远程执行	ai-native	MASTER/EXECUTOR、Pipeline/Stage、MCP 决策、job/result、心跳	其接口字段和部署假设
协议与沙盒	Codex	App Server、RPC、Policy、Sandbox、Thread Store、Trace	将内部协议直接作为公共 API

16.2 代码风格约束

- Agent Runtime 使用 Pi 的 TypeScript 状态化 Agent、事件流和显式工具执行风格；
- 扩展使用 DeepSeek Harness 的 ESM、注册效果、Service Definition/Provider/Consumer 分层；
- 任务平台遵循 Multica 的 Go 后端 + TypeScript/React 多端边界，服务端状态与客户端状态分离；
- 决策和执行服务遵循 ai-native 的 Python 服务、能力注册、job/result、心跳和适配器风格；
- 受控执行和协议边界遵循 Codex 的 Rust crate 分层、显式 Policy 和 Sandbox 端口；
- 新代码必须说明参考路径、复用模式、未复用部分、失败语义和测试边界；
- 如果参考工程没有对应实现，必须标记为新的架构决策，先讨论再编码。

16.3 真实源码依据索引

- Pi Loop：/Users/ninja/work/projects/brain/pi-mono/packages/agent/src/agent-loop.ts；
- Pi 状态与工具：/Users/ninja/work/projects/brain/pi-mono/packages/agent/src/agent.ts、harness/tools/、harness/session/；
- Pi 压缩：/Users/ninja/work/projects/brain/pi-mono/packages/agent/src/harness/compaction/；
- DeepSeek Harness
 - 核心：/Users/ninja/work/projects/brain/deepseek-harness/packages/core/、packages/llm/、packages/interaction/；
- Multica
 - 调度与运行时：/Users/ninja/work/projects/brain/multica/server/internal/dispatch/、daemon/、events/、realtime/；
- Multica 移动同步：/Users/ninja/work/projects/brain/multica/apps/mobile/data/、lib/use-ws-subscriptions.ts；
- ai-native 决策服务：/Users/ninja/work/projects/brain/ai-native/decision_service/；
- ai-native 执行器：/Users/ninja/work/projects/brain/ai-native/executor/；
- ai-native 后端：/Users/ninja/work/projects/brain/ai-native/backend/domain/、service_layer/、stages/；
- Codex App Server：/Users/ninja/work/projects/brain/codex/codex-rs/app-server/；
- Codex 策略与沙盒：/Users/ninja/work/projects/brain/codex/codex-rs/execpolicy/、linux-sandbox/、sandboxing/；

- Codex 持久化与回放 : /Users/ninja/work/projects/brain/codex/codex-rs/thread-store/、rollout-trace/。

17. 安全、可靠性与治理要求

17.1 安全

- Provider 能力最小化授权；
- 凭证不进入普通 Event 和模型上下文；
- Workspace、网络和进程访问有策略边界；
- System Agent 不能绕过人工门和审计；
- 高风险动作需要可见影响范围和人工接管。

17.2 可靠性

- 所有异步 Job 可取消、超时和恢复；
- Event 可去重、补发和重放；
- Task Graph 版本可回退；
- Artifact 和 Evidence 与 Task/Node/Session 可关联；
- 多设备视图最终一致，但控制平面状态迁移必须由单一拥有者确认。

17.3 可观测性

每次任务推进、Agent Turn、Tool Call、Provider Job、人工决定和 System Agent 介入都需要可以关联到 Task、Node、Session、Actor、来源和时间。指标、日志、Trace 和 Audit 互相补充，不能只保留自然语言日志。

18. L2 实现路线

阶段一：最小闭环

Task、简单 DAG、单 Agent Session、Pi 风格 Loop、一个 Model Provider、一个 Tool Provider、受控 Workspace、人工 Review Gate、Artifact/Evidence、Event 回写和跨端 Task View。

阶段二：多端与多 Agent

Device Sync、Agent Collaboration、Assignment/Handoff、Runtime 注册、Provider Catalog、重试和 Checkpoint。

阶段三：治理与加速

JEV 接入、System Agent 观察和介入、Memory Organizer、Evidence Challenge、Experience Ledger、升级/降级/回滚。

阶段四：规模化

多 Runtime 调度、Provider 市场、复杂 Graph Planner、跨 Workspace 协作、容量治理和多租户隔离。

每个阶段都必须先通过上一阶段的状态、事件、权限和恢复验证，不能只增加 Agent 数量而不增加证据和控制能力。

19. L2 设计评审清单

进入任何代码实现前，必须回答：

1. 这个需求属于哪个 L0 模块和哪个 L1 子模块？
2. 它改变的是 Command、Event、State、Artifact、Evidence 还是 Policy？
3. 哪个模块拥有状态迁移？
4. 失败、取消、超时、重试和回滚如何表达？
5. 哪个参考工程提供了相似实现？真实路径是什么？
6. 哪些部分是复用，哪些部分是本平台的新决策？
7. 是否绕过了 Human Gate、Policy Gate、Provider Gateway 或 Audit？
8. 是否会让原始 Evidence 不可追踪？
9. 如何进行单元、集成、回放和用户可见结果验证？
10. 是否需要更新 TypeScript、Go、Python 或 Rust 两侧的协议与文档？

20. 当前待确认问题

1. 第一版 Task Graph 是否只支持 DAG、显式 Retry 和人工门？
2. Event Store 是关系表追加日志，还是在第一版引入专用事件流？
3. Artifact 使用本地 Workspace、对象存储还是二者组合？
4. JEV 的边界是本地快速模型、规则执行器还是独立服务？
5. System Agent 的第一批 Career 只做观察与建议，还是允许有限自动介入？
6. 多 Agent 协作是否只允许 Task Node 级交接？
7. 哪些第三方 CLI 和 Provider 进入第一版白名单？
8. Human Gate 是否支持按风险等级调整确认强度？

这些问题影响 L2 的字段、协议和部署，但不应反向破坏本文件已经确定的 L0/L1 模块边界。

21. 实现契约总览

前面的章节定义模块和设计原则。本章开始进入可以交给工程团队拆解的实现契约。契约描述对象的语义、拥有者、变化方式和验证要求；它不绑定某一种数据库、消息队列或部署平台。

21.1 模块拥有权

对象或能力	唯一拥有者	其他模块的访问方式	禁止的访问方式
Task 与 Task State	Task Control Plane	Command、Query、Event	客户端直接写状态；Runtime 直接改 Task
Task Graph 版本	Task Control Plane	Planner Proposal、Graph Event	Agent 私自发布图版本
Agent Session	Agent Runtime	Runtime Intent、Session Event、Report	Control Plane 直接改 Loop 内部状态
Tool/Provider Job	Execution Environment	Execution Intent、Job Report	Agent 直接调用第三方 CLI
连接与订阅	Communication Layer	Subscribe、Publish、Resume	业务模块自行维护跨端连接
Event、Activity、Audit	Persistence & Event Foundation	Append、Projection、Query	以 Activity 代替原始事实
Artifact 与 Evidence	Persistence & Event Foundation	Artifact API、Evidence API	结果只保存在自然语言消息中
Policy 与 Human Gate	各自的拥有模块	Policy Check、Gate Decision	通过前端布尔值绕过检查
Agent Profile 与 Experience Ledger	System Agent Governance	Evaluation、Promotion Proposal	Agent 自己给自己升级

21.2 跨模块调用规则

跨模块调用分成四类：

1. Command：请求拥有者执行一次有副作用的动作；
2. Query：读取拥有者提供的视图，不改变事实；
3. Event：通知已经发生的事实，消费者不能把事件当成命令；
4. Port：同进程或跨进程都可以实现的稳定能力接口。

一个模块不能通过读取另一个模块的内部表来绕过

Port。早期单体实现可以共享数据库实例，但仍需要按拥有者划分表和写入入口。

21.3 统一请求与事件信封

以下是语义级结构，字段名和序列化格式在 L2 API 设计中确定：

```
RequestEnvelope
request_id    一次请求的幂等标识
actor_id     发起者
client_id    来源设备或 Agent Client
task_id      业务关联
node_id      可选的任务节点
session_id   可选的运行时上下文
command_type 命令类型
command_version 命令版本
payload      命令参数
cause_event_id 触发来源
```

EventEnvelope

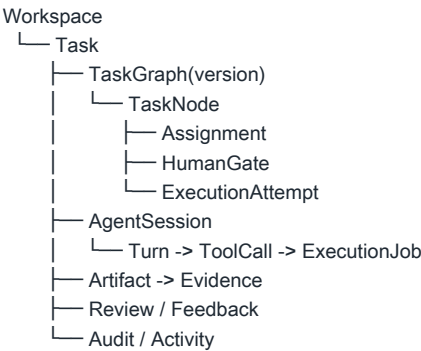
event_id	全局唯一事件标识
event_type	事实类型
event_version	事件版本
aggregate_type	Task、Session、Job、Artifact 等
aggregate_id	聚合标识
task_id/node_id	业务关联
actor_id/source	产生事件的主体和模块
occurred_at	事实发生时间
recorded_at	进入持久化的时间
causation_id	直接原因事件或命令
correlation_id	一次业务链路的关联标识
payload	事实数据
evidence_refs	支持该事实的证据引用
schema_version	信封版本

21.4 写入与读取的一致性

- 拥有状态的模块在同一事务中写入状态变化和本模块产生的领域事件；
- 事件投影可以异步执行，客户端读取的是带版本号的 Read Model；
- 对需要立即判断的控制动作，使用拥有者的同步 Command 响应；
- 对日志、通知、时间线和治理观察，使用异步 Event；
- 任何重试都必须携带原始 request_id 或新的明确尝试编号；
- 事件消费者必须幂等，重复事件不能重复产生副作用。

22. 核心数据模型与关系

22.1 对象关系



对象之间使用稳定 ID 和版本引用连接。自然语言标题、展示名称和摘要不能作为关联键。

22.2 Task 与 Node 的字段语义

对象	必须表达的语义	由谁写入	由谁读取
Task	目标、范围、状态、图版本、参与者、风险级别	Control Plane	所有模块的受限视图
TaskNode	节点目标、依赖、能力要求、责任主体、状态	Control Plane	Planner、Runtime、Human、System Agent
Assignment	指定主体、租约、能力、接管人、尝试次数	Control Plane	Runtime、Communication、Audit
HumanGate	待决定对象、允许动作、风险、截止时间、决定	Control Plane/Human	Client、System Agent、Audit
ExecutionAttempt	尝试编号、Session、Job、开始/结束、结果	Runtime/Execution	Control Plane、Persistence、Governance

22.3 Session、Turn、ToolCall 与 Job

Session
state: Created | Running | WaitingInput | Paused | Cancelling | Completed | Failed
current_turn_id
context_revision
checkpoint_id

Turn
input_refs
model_request
assistant_output
tool_call_refs
stop_reason

ToolCall
tool_name
validated_args
policy_decision
execution_job_id
result_ref

ExecutionJob
provider_id
workspace_id

```
state: Queued | Running | Succeeded | Failed | Cancelled | TimedOut
retry_attempt
artifact_refs
evidence_refs
```

Session 记录运行时上下文，Turn 记录一次模型回合，ToolCall 记录模型发出的能力请求，Job 记录真正的受控执行。四者必须分开，否则无法解释“模型想做什么”和“系统实际做了什么”的差异。

22.4 Artifact、Evidence 和 Provenance

Artifact 表示产物，Evidence 表示产物或观察支持的判断，Provenance 表示来源链。一个 Artifact 可以支持多个 Evidence，一个 Evidence 可以引用多个 Artifact 和 Event。

```
Artifact
artifact_id, kind, uri, digest, media_type, size
task_id, node_id, session_id, created_by
version, parent_artifact_id, retention_policy
```

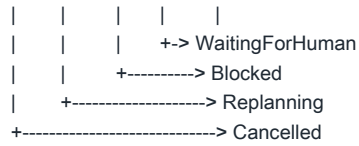
```
Evidence
evidence_id, kind, claim, source_refs
confidence, verifier, verified_at
derived_from, supersedes, status
```

派生摘要必须通过 derived_from 指向原始来源；删除或过期只改变可见性和保留策略，不制造“从未发生过”的假象。

23. 状态机设计

23.1 Task 状态机

Draft -> Planned -> Ready -> Running -> Review -> Completed



Failed -> RetryPending -> Ready

Failed -> Replanning

Running/Review -> RolledBack

迁移条件：

- Draft -> Planned：Graph Proposal 已通过结构校验；
- Planned -> Ready：所有前置依赖满足，Assignment 已确认；
- Ready -> Running：Runtime Intent 已发出且获得执行租约；
- Running -> WaitingForHuman：Runtime 或 System Agent 产生人工门；
- Running -> Review：节点结果和必要 Evidence 已回写；
- Review -> Completed：验收条件满足且没有待处理 Gate；
- 任意可运行状态 -> Cancelled：拥有权限的 Actor 发出取消命令；
- Failed -> RetryPending：错误属于可重试类别且重试预算未耗尽；
- Failed -> Replanning：原计划、依赖或能力假设失效；
- RolledBack：恢复点和影响范围已经记录。

23.2 Task Node 状态机

Pending -> Ready -> Claimed -> Executing -> AwaitingResult

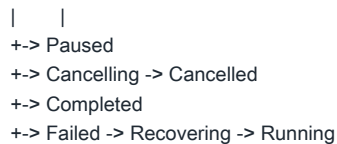


AwaitingResult -> Succeeded | Failed | NeedsHuman | NeedsReplan

Node 状态由 Control Plane 拥有。Runtime 只能报告 ExecutionStatus，不能直接写 Node 状态。

23.3 Agent Session 状态机

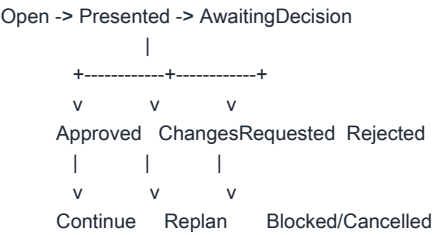
Created -> Starting -> Running -> WaitingInput -> Running



Session 恢复需要 Checkpoint、最后确认的 Event 序列和未完成 Job

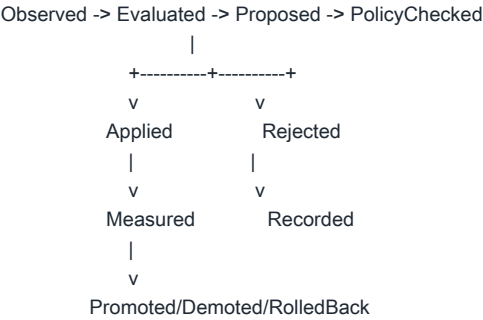
列表。恢复不是重新发送最后一句提示词，而是从可验证的运行时状态继续。

23.4 Human Gate 状态机



人工 Gate 的决定必须绑定 Actor、Client、输入版本、展示的 Evidence 和决定时间。

23.5 System Agent 介入状态机



System Agent 的建议在 Applied 之前不改变业务事实；自动介入也必须产生 Audit 和撤销路径。

24. 关键时序与跨模块流程

24.1 创建并执行一个 Task

Human Client -> Interaction Gateway: CreateTask
 Gateway -> Control Plane: CreateTask Command
 Control Plane -> Event Foundation: TaskCreated
 Control Plane -> Planner: BuildGraph Proposal
 Planner -> Control Plane: GraphProposed
 Control Plane -> Control Plane: Validate and Publish Graph
 Control Plane -> Runtime: ExecutionIntent
 Runtime -> Communication: SessionStarted / Progress
 Runtime -> Provider Gateway: ToolRequest
 Provider Gateway -> Execution Environment: ExecutionJob
 Execution Environment -> Persistence: Artifact / Evidence / JobResult
 Runtime -> Control Plane: RuntimeReport
 Control Plane -> Human Client: ReviewGate
 Human Client -> Control Plane: Approval or ChangesRequested
 Control Plane -> Persistence: StateChanged / Audit

24.2 设备断线恢复

1. Client 记录最后确认的 Event Sequence ；
2. Connection Manager 检测连接失效并关闭旧租约 ；
3. Client 使用 Actor、Client、Task 和最后序列重新订阅 ；
4. Communication Layer 返回当前 Task View 版本和可补发事件范围 ；
5. Client 先替换读模型，再应用未确认事件 ；
6. 重复事件通过 Event ID 去重 ；
7. 无法补发时返回 SnapshotRequired，而不是静默丢失。

24.3 Agent-to-Agent 交接

1. Agent A 在当前 Node 产生 Handoff Proposal ；
2. Control Plane 检查 A 的 Assignment 和交接权限 ；
3. Handoff 写入输入 Artifact、Evidence、未解决问题和目标 Agent ；
4. Communication Layer 只传递带 Task/Node 引用的交接消息 ；
5. Agent B 接受或拒绝，并产生新的 Assignment ；
6. Control Plane 更新 Node 责任主体 ；
7. Audit 记录交接原因、双方版本和结果。

24.4 记忆异步整理

1. Context Manager 发现接近上下文预算 ；
2. JEV 或 Memory Organizer 读取可整理的派生上下文 ；
3. 生成压缩候选、删除候选、冲突候选和来源列表 ；
4. Policy 检查候选是否会影响任务事实或高风险判断 ；
5. 低风险候选写入新 Memory Revision ；
6. 高风险候选进入 System Agent 或 Human Gate ；
7. 原始事件和 Evidence 保留，Session 使用新版本继续。

25. 代码仓库与包边界蓝图

以下是目标平台的拟议代码结构。它是对参考工程组织方式的融合，不表示现有工程已经存在这些目录。

platform/	
apps/	
web/	Web 端入口
desktop/	Desktop 壳和本地能力
mobile/	Mobile 端入口
cli/	CLI 和自动化入口
packages/	
contracts/	稳定协议、ID、事件信封、错误码
task-control/	Task、Graph、Assignment、Gate
agent-runtime/	Session、Loop、Context、Provider 端口
communication/	Connection、Subscription、Message
execution/	Provider、Job、Policy、Workspace
persistence/	Event、State、Artifact、Evidence、Audit
collaboration/	Human-Agent 协同状态机
system-agents/	Career、Sequence、Evaluation、Intervention
projections/	Task View、Activity、Timeline、Inbox
shared/	时间、错误、日志、配置和测试工具
services/	
api-gateway/	外部 HTTP/RPC 入口
control-plane/	Task Control 应用服务
runtime-host/	Agent Runtime Host
execution-host/	Provider 和 Sandbox Host
event-worker/	投影、通知、治理异步任务
system-agent-worker/	System Agent 观察和介入
adapters/	
pi/	Pi Runtime adapter
dsh/	DeepSeek Harness adapter
multica/	Multica task/realtime adapter
ai-native/	MASTER/EXECUTOR adapter
codex/	App Server/Policy/Sandbox adapter
infrastructure/	
sql/	SQLite/Postgres
blob/	本地或对象存储
queue/	事件与任务队列
telemetry/	metrics/logs/trace
docs/	
adr/	Architecture Decision Records
protocols/	协议与事件文档
runbooks/	故障恢复和运维手册

25.1 依赖方向

apps -> api-gateway -> application -> domain
-> ports <- adapters/infrastructure

Domain 不依赖应用框架和基础设施。Application 负责事务、权限、幂等和流程编排。Adapters 负责把端口接到 Pi、Provider、数据库或外部协议。

25.2 多语言边界

- TypeScript : Agent Runtime、协议客户端、Web/Desktop/Mobile、实时交互和共享类型；
- Go : 高并发任务服务、Runtime 注册、调度、事件入口和后台服务，可参考 Multica；
- Python : 决策服务、评估器、实验性 System Agent 和 ai-native 风格的 Executor；
- Rust : 高信任的协议、沙盒、Policy、进程和安全边界，可参考 Codex。

第一版不要求四种语言同时出现。应该先选择能完成最小闭环的语言组合，再通过稳定的协议隔离后续迁移。

26. API、事件和错误设计蓝图

26.1 外部 API 分组

API 组	负责内容	不负责内容
Task API	创建、查询、命令、Graph 和 Gate	直接调用模型或 Shell
Runtime API	Session、Intent、Progress、Report	修改 Task 状态
Communication API	订阅、恢复、Agent 交接	业务规则判断
Provider API	能力目录、Job、结果和健康	任务规划
Artifact API	上传、下载、版本、引用和 Evidence	允许性判断
Governance API	评估、介入、Profile 和审计	隐式修改事实

26.2 命令命名

命令使用动词和明确对象，例如 CreateTask、PublishGraph、AssignNode、ApproveGate、StartSession、RequestExecution、CancelJob、CreateMemoryRevision。命令响应只确认接收、拒绝或已经完成的事实，不返回未经标记的推理结论。

26.3 事件命名

事件使用过去时，例如 TaskCreated、GraphPublished、NodeAssigned、SessionStarted、ToolCallRequested、ExecutionCompleted、HumanGateApproved、ArtifactRecorded、InterventionApplied。事件不使用模糊的 Updated 覆盖多个事实。

26.4 错误分类

InvalidRequest	参数或版本不合法，不应重试
NotAuthorized	权限不足，通常需要人工或策略调整
Conflict	版本冲突，可读取新版本后重新规划
PolicyDenied	策略拒绝，不自动重试
CapacityUnavailable	暂无执行资源，可退避重试
ProviderFailure	第三方能力失败，按 Provider 策略重试
Timeout	超时，必须区分是否已经产生副作用
Cancelled	主动取消，保存取消来源
EvidenceMissing	结果缺乏必要证据，进入 Review 或 Replan
InternalFailure	平台内部异常，进入恢复和告警流程

错误对象必须带 retryability、owner、stage、cause_refs、attempt 和 recovery_hint。自然语言 message 只用于展示，不能承担机器判断。

27. 持久化、并发和恢复设计

27.1 第一版存储建议

第一版可以采用关系数据库保存 Task、Node、Assignment、Gate、Session 索引、Event、Activity、Audit 和 Projection；Workspace 文件或对象存储保存 Artifact；Event Worker 负责异步投影和通知。

选择关系存储的理由是：Task Graph、Assignment、Gate 和审计查询需要明确关联、事务和版本约束；不应为了“事件驱动”而过早引入多个基础设施。

27.2 并发控制

- Task 和 Node 使用版本号或 compare-and-swap 防止旧客户端覆盖新状态；
- Assignment 使用租约和 fencing token 防止旧 Worker 继续写结果；
- Event Append 使用唯一 Event ID 和 aggregate version；
- Job 使用 attempt number 和 provider job ID；
- Projection 记录最后处理的 Event Sequence；
- Human Gate 记录输入版本，过期决定必须重新展示。

27.3 恢复顺序

1. 读取最后确认的 Task/Node/Session 状态；
2. 找到未完成的 Intent、Job、Gate 和 Projection；
3. 通过 Event/Checkpoint 判断每个动作是否已经产生副作用；
4. 对不确定的外部 Job 查询 Provider 状态，不直接重复执行；
5. 对可安全重试的动作生成新的 Attempt；
6. 对无法判断的动作进入人工复核或治理队列；
7. 恢复完成后写入 RecoveryCompleted 和 Audit。

27.4 数据保留

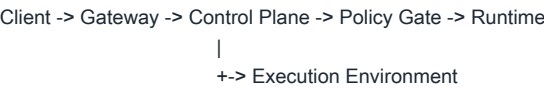
原始 Audit、Evidence 和已确认事实按合规策略保留；派生摘要、缓存和通知可以设置更短 TTL。删除或脱敏操作必须产生 Audit，并保留删除原因、操作者、范围和版本。

28. 安全模型与权限矩阵

28.1 权限主体

权限主体包括 Human Actor、Task Agent、System Agent、Provider、Runtime Host、Client 和后台 Worker。每个主体拥有范围化能力，不能仅凭名字或 Agent 类型获得权限。

28.2 权限检查位置



客户端检查只用于体验；真正的 Policy Check 必须在拥有状态或能力的一侧再次执行。System Agent 的建议不能作为权限本身。

28.3 最小权限矩阵

动作	Human	Task Agent	System Agent	Provider
创建 Task	按 Workspace	受委托	受策略	否
发布 Graph	受角色	提案	评估/建议	否
调用 Tool	受确认/策略	受 Assignment	建议/阻止	执行
修改 Workspace	受策略	受 Job 权限	否	受 Job 权限
通过 Human Gate	是	否	否，除非明确自动门	否
修改原始 Evidence	否	否	否	否
升级 Profile	依据评估	不能自升	提案	否

28.4 凭证和敏感信息

凭证只能在 Provider 或 Execution Environment 的最小作用域中短暂注入。凭证值不进入普通 Event、Artifact、Prompt、Trace 或错误 message。需要记录的是凭证类型、版本和使用结果，而不是秘密本身。

29. 可观测性、审计和解释

29.1 三类记录

- Log：开发和运维诊断；
- Trace：一次跨模块请求的时序和耗时；
- Audit：对权限、事实、人工决定和治理介入的长期记录。

三者可以共享 correlation ID，但保留周期、访问权限和内容不同。

29.2 必须可解释的动作

Task 状态迁移、Graph 发布、Assignment、Tool Call、Provider Job、Human Gate 决定、System Agent 介入、Profile 升降级和 Rollback 都必须能够回答：谁做的、基于什么、影响什么、结果是什么、如何撤回。

29.3 指标

第一版至少记录：Task 完成率、计划重做率、人工等待时长、Runtime 成功率、Tool Provider 失败率、平均恢复时间、Evidence 缺失率、重复执行率、System Agent 误报率和人工接管率。

Agent 的 token、模型延迟和工具耗时是执行指标，不能直接等价于任务价值；最终指标仍应回到 Task 结果和证据质量。

30. 测试与验收矩阵

30.1 单元测试

- Task/Node 状态迁移和 Graph 校验；
- Assignment 租约和 fencing；
- Policy 规则与风险分类；
- Agent Loop 的工具调用、停止和错误分支；
- Context 压缩的来源保留和版本化；
- Event 去重、投影和重放；
- Provider Adapter 参数校验和结果转换；
- System Agent 评价、升级、降级和回滚。

30.2 集成测试

- Create Task -> Plan -> Assign -> Execute -> Review -> Complete；
- Runtime 断线恢复；
- Provider 超时、取消和重复提交；
- 多设备订阅和序列补发；
- Agent-to-Agent Handoff；
- Human Gate 过期、请求修改和接管；
- 原始 Evidence 与派生 Memory 的追踪；
- System Agent 介入后业务事实和 Audit 一致。

30.3 回放与故障注入

使用保存的 Event、Artifact 元数据和 Provider 模拟器重放任务。注入断线、进程崩溃、重复 Event、数据库冲突、Provider 超时、策略拒绝和人工延迟，验证系统最终状态、Audit 和恢复路径。

30.4 验收标准

第一版完成的判定不是“Agent 能回答问题”，而是：一个 Task 能创建、拆解、分派、执行、请求人工、回写 Artifact/Evidence、跨设备查看、从中断恢复，并能解释每次状态变化。

31. 架构决策记录 ADR 索引

后续每个重大决策都使用 ADR 记录：背景、问题、候选方案、选择、取舍、影响、迁移和回滚。建议首批 ADR：

1. ADR-001：Task 作为唯一协作主对象；
2. ADR-002：Task Graph 第一版采用有限 DAG；
3. ADR-003：Control Plane 与 Agent Runtime 分离；
4. ADR-004：第三方能力必须经过 Provider Gateway 和 Execution Environment；
5. ADR-005：原始 Evidence 不可无痕修改；
6. ADR-006：Agent-to-Agent 使用 Task Node 级交接；
7. ADR-007：JEV 作为 Agent Loop 内部加速器；
8. ADR-008：System Agent 采用 Career/Sequence/Instance；
9. ADR-009：第一版采用关系状态存储 + Event/Activity + Artifact Store；
10. ADR-010：第一版人机协同必须支持暂停、恢复、请求修改和接管。

ADR 不替代本文件，而是记录某个选择为什么成立、什么时候需要重新评估。

32. 从架构到代码的执行顺序

第一步：冻结共享契约

先实现 contracts 中的 ID、RequestEnvelope、EventEnvelope、错误分类和版本规则；不先写 UI 或 Agent Prompt。

第二步：实现 Task Control 最小闭环

完成 Task、Node、有限 DAG、Assignment、Human Gate、状态迁移和 Event Append。用假 Runtime 验证控制平面。

第三步：接入 Pi Runtime

实现 Session、Agent Loop、Tool Call、Runtime Report 和一个受控 Tool Provider。保持 Pi 的 Loop 事件顺序和显式状态。

第四步：接入 Execution Environment

先接本地 Workspace 和一个 CLI/文件 Provider，再接沙盒、凭证和远程 Executor。每个 Provider 都需要能力描述、Policy Check 和结果 Evidence。

第五步：接入跨端和人机闭环

实现 Task View、Device Sync、Review Gate、Pause/Resume、Request Changes 和 Take Over。

第六步：加入持久化恢复

补齐 Event Replay、Checkpoint、Artifact/Evidence、Audit、断线恢复和故障注入。

第七步：加入 System Agent 和 JEV

先做只观察和建议，再开放有限自动介入；先测量误报、延迟、人工接管和回滚，再决定升级权限。

第八步：逐模块映射真实源码

每个实现 PR 必须在设计文档中填写：L0/L1/L2 位置、参考工程路径、采用的代码模式、未采用的部分、新增 ADR、测试和回滚方案。

33. 完整性检查表

一份模块设计只有同时回答以下问题，才算达到可实现深度：

- 它的状态由谁拥有？
- 它接收什么 Command、Query 和 Event？
- 它产生哪些状态、Event、Artifact 和 Evidence？
- 它如何处理重试、取消、超时、冲突和恢复？
- 它如何进行 Policy、Human Gate 和权限检查？
- 它如何与其他模块通过 Port 连接？
- 它的代码目录和参考工程哪一部分对应？
- 它的数据库、队列、缓存和文件边界是什么？
- 它的日志、Trace、Audit 和指标是什么？
- 它如何测试正常流程、失败流程、恢复流程和回放？
- 它的设计哲学如何转化成可检查的不变量？
- 它尚未决定的部分是否明确标记为待定，而不是伪装成既有实现？

34. 优秀设计文档方法与本文件的对应关系

本文件采用多种成熟设计文档方法的交集，不把任何一种方法当作唯一模板。Google Design Doc 负责把问题、目标、非目标、候选方案、决策、风险和发布计划写清楚；C4 负责把 Context、Container、Component 和 Code 视图分层；arc42 负责补足约束、运行时、部署、质量属性和风险；RFC/ADR 负责记录候选方案和不可逆决策；Well-Architected 负责把可靠性、安全性、成本、性能和可运维性变成可检查的质量属性。

34.1 方法对照

方法	解决的问题	在本文件中的落点	评审产物
Google Design Doc	为什么做、做什么、不做什么	第 2、6、17、18、40 节	目标、非目标、风险、发布计划
C4	读者如何逐层理解架构	第 4、36 节以及架构图目录	Context、Container、Component、Runtime 视图
arc42	如何完整描述工程架构	第 3、5、17、27、28、37、38、39 节	约束、策略、数据、部署、质量、风险
RFC/ADR	候选方案如何比较和落档	第 20、31、42 节	ADR、决策表、迁移和回滚
Well-Architected	质量如何量化和持续验证	第 17、27、29、37、40 节	SLO、指标、故障演练、成本预算

34.2 本文件的评审顺序

评审先确认问题边界和非目标，再确认 L0 模块边界；随后按 C4 的粒度检查 L1 子模块、运行时和部署视图；最后检查 L2 交付物、测试、指标、权限和回滚。任何只给出组件名称、没有状态拥有者、输入输出、失败路径和验收证据的内容，都不能视为完成的架构设计。

34.3 设计文档的固定问题

每次变更都必须回答：当前问题是什么？哪个用户或系统受到影响？为什么现有参考工程不足以直接复用？有哪些候选方案？选择的方案牺牲了什么？如何迁移？如何回滚？如何证明它按预期工作？这些问题分别映射到目标、边界、依据、ADR、风险、发布和测试章节。

35. 需求追踪矩阵

需求追踪矩阵把用户目标连接到架构原则、L0 模块、L1 子模块、L2 交付物和验收测试。它是变更评审的入口，不是把所有实现细节提前写进 L0。

用户目标	架构原则	L0 模块	L1 子模块	L2 交付物	验收证据
人和 Agent 围绕同一个任务协作	Task 是唯一协作主对象	Task Collaboration & Control Plane	Task Model、Graph、Assignment、Human Gate	状态迁移、命令、事件和并发规则	创建到完成的端到端回放
多端看到一致任务进展	客户端只提交 Command，事实由服务端产生	Human & Device Experience、Communication	Device Sync、Query、Activity Projection	游标、补发、冲突和离线策略	两设备断线恢复测试
Agent 能持续执行并恢复	Runtime 拥有执行上下文，Control Plane 拥有推进权	Agent Runtime、Persistence	Session、Loop、Checkpoint、Context	Loop 事件、恢复协议、停止语义	进程崩溃后继续执行
安全调用第三方能力	Provider 必须经过 Policy 和执行边界	Execution Environment	Provider Gateway、Policy Gate、Executor	能力描述、凭证引用、超时、取消	越权、超时、重复提交测试
多 Agent 可交接而不丢责任	交接基于 Node、Assignment、Artifact、Evidence	Communication、Control Plane	Handoff、Inbox、Lease	交接协议、幂等键、责任链	Agent A 到 B 的交接回放
记忆可整理但事实可追溯	原始 Evidence 不可无痕修改	Persistence & Event、System Agent	Memory、Evidence、Audit、Intervention	版本、来源、降权、回滚	原始证据与派生记忆比对
Agent 能力可评价和进化	进化必须受约束、可解释、可回滚	System Agent & Governance	Career、Sequence、Instance、Evaluation	评分、门槛、升级、降级、回滚	误评估和回滚演练

矩阵中“验收证据”必须能在测试、回放、指标或审计记录中找到。没有证据的需求只能标记为待设计，不能在发布说明中写成已支持。

36. 架构视图目录

同一系统需要面向不同读者提供不同视图。视图之间共享对象和边界，但不要求每张图显示全部组件。当前架构图目录提供 L0 和 L1 的静态视图；本节规定后续 L2 图和评审材料的组织方式。

36.1 Context View：系统上下文

展示 Human、Desktop/Mobile/CLI Client、Agent、System Agent、第三方 Provider、Workspace 和外部身份/凭证系统，以及本平台的责任边界。该图回答“谁与平台交互、平台依赖什么、平台不控制什么”。

36.2 Container View：可部署容器

展示 API Gateway、Task Control Plane、Agent Runtime Host、Execution Host、Event Worker、System Agent Worker、Artifact Store、SQL State Store、Event Store、Telemetry 和客户端。容器是部署和进程边界，不等于代码包，也不预设必须拆成微服务。

36.3 Component View：模块内部组件

每个 L1 模块至少提供一张 Component View，标出领域对象、应用服务、端口、适配器、状态拥有者和事件出口。Component View 必须能回指第 25 节的代码包边界和第 16 节的源码依据。

36.4 Runtime View：关键运行时场景

至少维护以下时序：创建并执行 Task、人工 Gate、Provider 超时、设备断线恢复、Agent-to-Agent Handoff、异步 Memory 整理、System Agent 介入和回滚。每条时序都要标明 Command、Event、Checkpoint、重试边界和最终责任主体。

36.5 Deployment View：部署拓扑

描述客户端、网关、控制平面、运行时主机、执行主机、事件工作器、治理工作器和数据基础设施的网络、身份、伸缩和故障域。部署图不能反向定义领域边界；如果部署拆分改变了状态拥有关系，必须新增 ADR。

36.6 Data/Information View：数据与信息流

展示 Task、Node、Session、Event、Artifact、Evidence、Audit、Memory 和 Capability 的生命周期、来源、版本、保留期限和访问级别。重点标出原始事实、派生视图和可删除缓存之间的关系。

36.7 Security View：安全边界

展示用户身份、Agent 身份、System Agent 身份、Provider 凭证、Workspace 权限、Policy Gate、Human Gate、Audit 和密钥边界。所有跨边界箭头都应映射到一个权限检查点和一类可审计事件。

37. 质量属性目标与验证方式

质量属性先定义目标和测量方式，再决定技术实现。以下是第一版的基线目标；具体数值可以在压测和真实使用数据后调整，但调整必须通过 ADR。

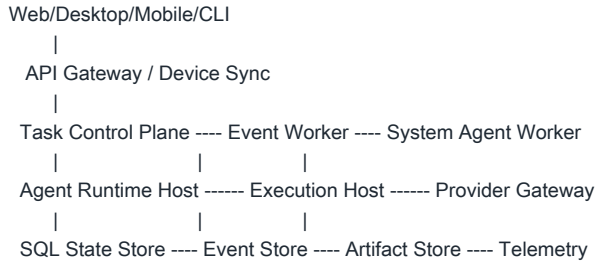
质量属性	第一版目标	测量方式	主要责任模块
可用性	控制平面、事件追加和任务查询具备单实例故障后的恢复路径	可用性探针、故障注入、恢复时间	Control Plane、Persistence
交互延迟	Command 接收后快速返回受理结果；长任务通过事件异步更新	p50/p95/p99 延迟、端到端 Trace	Communication、Control Plane
可恢复性	Session、Assignment、Provider Job 可从 Checkpoint 或幂等键恢复	崩溃、断线、重复提交测试	Runtime、Execution、Persistence
可审计性	高风险动作可关联 Actor、输入、Policy、结果和回滚	Audit 完整性检查、事件回放	Audit、Policy、System Agent
安全性	最小权限、凭证隔离、沙盒和人工门覆盖高风险动作	越权、注入、泄露和逃逸测试	Execution、Policy、Human Gate
可扩展性	新 Provider、新客户端和新 System Agent 不改变 Task 核心模型	适配器契约测试、兼容性测试	Provider、Communication、Governance
可观测性	任务、Session、Tool Call、Provider Job 共享 Trace/Correlation ID	Trace 覆盖率、日志关联率	Telemetry、Event
成本控制	能测量模型、工具、执行环境和人工等待的成本	费用计量、预算告警、任务成本报告	Runtime、Provider、Control Plane

每个质量属性必须至少绑定一个指标、一个测试场景和一个故障处理动作。只有描述“高性能”“高可靠”而没有测量方式的要求，不进入验收清单。

38. 部署拓扑与运行边界

38.1 第一版拓扑

第一版可以从模块化单体或少量进程开始，但保留以下逻辑边界：



上图是逻辑拓扑。实际部署可以合并进程，但必须保留请求、事件、权限和状态拥有关系。Runtime Host 不直接修改 Task 状态；Execution Host 不持有长期用户凭证；Worker 不能绕过事件追加接口写入事实表。

38.2 客户端与设备

客户端保存短期 UI 状态、订阅游标和待发送 Command，可缓存最近的 Activity 和 Artifact 元数据。客户端不能把本地缓存当作权威事实；重连后必须按游标补发 Event 或重新获取投影。

38.3 工作器与故障域

Event Worker 负责投影和通知，System Agent Worker 负责观察和治理，二者都应可重复消费。Runtime Worker 和 Provider Job 需要租约、心跳、超时和 fencing，避免旧进程在恢复后继续写入。

38.4 数据基础设施

SQL State Store 保存当前状态和索引；Event Store 保存追加事实；Artifact Store 保存大对象及其校验和；Telemetry 保存日志、指标和 Trace。第一版可以使用同一数据库实例，但逻辑表、权限和保留策略必须分开。

39. 威胁模型与防护要求

威胁模型采用“资产 - 入口 - 影响 - 控制 - 证据”的格式。安全控制必须落在调用链上，不能只写在部署说明里。

威胁	受影响资产	防护控制	必须保留的证据
Agent 越权调用工具	Workspace、凭证、外部系统	Capability Allowlist、Policy Gate、沙盒、人工门	Policy Decision、Tool Call、Audit
Prompt Injection	Task 上下文、工具参数、Memory	来源标记、上下文隔离、参数校验、人工复核	原始输入、风险评分、拒绝原因
Provider 供应链风险	执行主机、Artifact、凭证	Provider 签名/版本、最小权限、网络出口限制	Provider Manifest、Hash、Job 记录
凭证泄露	API Key、OAuth Token、Workspace Secret	Secret Broker、短令牌、脱敏日志、轮换	访问审计、轮换事件
恶意 Artifact	用户工作区、后续 Agent	类型和大小限制、病毒/脚本扫描、隔离预览	Artifact Hash、扫描结果
Event 重放或篡改	Task 状态、通知、Audit	Event ID 幂等、顺序检查、签名或校验和	Replay 检测、拒绝记录
System Agent 误介入	Memory、计划、Agent Profile	建议与执行分离、影响范围限制、人工门、回滚	Intervention、Before/After、Rollback
Memory 污染	后续决策上下文	来源分级、原始证据保留、版本化和降权	Memory Provenance、版本链
人工确认欺骗	高风险操作	明确展示影响、来源、差异和撤回方式	Human Gate 输入、展示版本、决定

第一版不允许 System Agent 直接删除原始 Evidence、绕过 Human Gate 或提升自己的权限。任何“暗面”能力都只能作为风险识别、反例生成和负反馈建议，并受相同 Policy、Audit 和回滚约束。

40. 发布、运维与故障演练

40.1 配置与版本

模型、Provider、工具权限、System Agent 规则和 Feature Flag 都必须版本化。运行中的 Task 记录所使用的版本引用；配置变更不得静默改变正在执行的 Session 语义。

40.2 灰度与回滚

Runtime、Provider、System Agent Policy 和事件投影都采用小范围灰度。灰度指标至少包含成功率、人工接管率、误报率、延迟、成本和回滚次数。回滚优先恢复到上一个已验证版本，不能通过删除事件或覆盖 Audit 来“修复”历史。

40.3 Schema 与 Event 迁移

新增字段优先采用向后兼容的可选字段；事件类型和版本必须能被旧消费者识别或安全忽略。破坏性迁移需要双写/双读窗口、回放验证和明确的终止条件。

40.4 故障演练

第一版至少演练：Control Plane 崩溃、Runtime 进程被终止、Provider 超时、Event Worker 重复消费、数据库只读、Artifact Store 不可用、移动端离线、Human Gate 过期和 System Agent 误介入。每次演练记录检测时间、恢复时间、数据损失、人工动作和后续 ADR。

40.5 SLO 与告警

告警按用户影响分级：任务无法创建或恢复为最高等级；单个 Provider 失败为局部等级；投影延迟、成本异常和 System Agent 误报为治理等级。告警必须链接到 Runbook、最近 Trace 和可执行的降级动作。

41. 文档治理与变更流程

41.1 文档版本

文档版本分为架构基线版本和实现版本。架构基线只在 L0 边界、共享对象、关键不变量或质量目标变化时升级；普通实现细节应通过 L2 文档、代码评审和 ADR 管理。

41.2 代码路径校验

每个“已实现依据”必须包含仓库、相对路径、符号或目录和核对日期。每次代码目录重命名后，必须在同一个变更中更新本文件的引用，不能留下无法打开的旧路径。

41.3 图文一致性

架构图只展示本层级需要的抽象。L0 图不列字段和代码组件；L1 图展示子模块和边界；L2 图才展示接口、状态、数据结构和部署细节。图中的每个模块必须能在本文找到责任、输入输出和参考依据。

41.4 评审角色

架构评审至少包含：领域/产品代表、Control Plane 负责人、Runtime/Execution 负责人、客户端与通信负责人、数据与可靠性负责人、安全与治理负责人。涉及 System Agent 的变更还需要对误介入、回滚和人类能动性进行专门评审。

41.5 变更流程

需求或事故 -> 影响范围 -> 候选方案 -> ADR
-> 文档与图件更新 -> 契约/测试更新
-> 代码实现 -> 回放/故障演练 -> 发布或回滚

变更完成的最低条件是：需求追踪矩阵有对应行、至少一个架构视图更新、相关 ADR 已落档、测试和监控已补齐、回滚路径已验证。

42. 参考实现边界与证据等级

为了避免把设计愿景误写成已有能力，所有参考工程依据采用三档证据：

证据等级	定义	文档写法
A：直接实现	源码和测试中存在可运行的对应逻辑	“参考并复用其边界/模式”，附路径
B：局部模式	存在相似机制，但对象或边界不同	“借鉴其模式”，说明差异
C：架构启发	只有理念或局部线索，不能直接复用	“作为设计启发”，标记待设计

42.1 当前工程的证据边界

- pi-mono 对 Agent Loop、Session、Tool、Compaction 和 Harness 提供 A 级实现依据；JEV 加速、跨任务调度和本平台的 System Agent 仍是本平台设计。
- deepseek-harness 对交互、LLM 调用和 Harness 组织提供 A/B 级模式；不能据此声称已经实现通用 Task Control Plane。
- multica 对 Dispatch、Daemon、Event、Realtime 和移动端数据同步提供 A 级模式；不能直接替代本平台的 Task Graph 和 Human Gate。
- ai-native 对决策服务、Pipeline/Stage、执行记录、人工输入和阶段产物提供 A/B 级模式；通用 Artifact Store、全局 Audit 和 Agent-to-Agent 协议仍需本平台设计。
- codex 对 App Server、执行策略、沙盒、线程存储和 Trace 提供 A 级工程参考；其产品语义和权限边界不能未经适配直接移植。

42.2 代码落地规则

实现 PR 必须注明：采用哪一档证据、参考的具体路径、保留了哪些代码风格、哪些部分因领域差异没有复用、对应的 ADR 和新增测试。没有路径和测试的“参考某工程”不算实现依据。

43. L2 深化模板与下一轮输入

后续逐个模块进入 L2 时，使用以下固定模板，避免重新发明文档结构：

1. 模块目标、非目标和状态拥有者；
2. 领域对象、字段语义、生命周期和不变量；
3. Command、Query、Event、Port 和 Adapter 契约；
4. 正常流程、并发流程、失败流程、取消流程和恢复流程；
5. Policy、Human Gate、权限、凭证和审计；
6. 数据库表、索引、队列、缓存、Artifact 和保留策略；
7. 代码目录、依赖方向、语言边界和参考工程路径；
8. 指标、日志、Trace、告警和 Runbook；
9. 单元、集成、回放、故障注入和安全测试；
10. ADR、迁移、灰度、回滚和未决问题。

下一轮 L2 设计建议优先选择 Task Collaboration & Control Plane 或 Agent Runtime。前者先冻结 Task/Node/Assignment/Gate 的契约，后者再以 Pi 的 Loop/Session/Tool 顺序接入 JEV 和 Provider；两者都必须先完成本模板中的状态、事件、恢复和测试部分，再讨论更细的代码实现。

44. 第一版复杂度预算与最小公共模型

复杂度预算是第一版的硬约束。它限制对外可见的状态、字段和子模块数量；内部仍可以保留诊断所需的阶段、错误码和事件。对外模型越小，跨端同步、审计、迁移和人工理解就越稳定。

44.1 状态压缩规则

对外状态采用“主状态 + reason_code + revision”三元组。主状态表达用户和控制平面真正关心的生命周期，reason_code 表达失败、等待、取消、重试和重新规划的具体原因，revision 用于并发和同步。内部阶段、工具步骤和调度信号只写入 Event，不扩散到所有 API。

对象	第一版对外主状态（5-7 个）	内部细节如何保留
Task	Draft、Planned、Active、Waiting、Review、Completed、Closed	reason_code 区分 Blocked、Failed、Cancelled、Replanning、RolledBack
Task Node	Pending、Ready、Active、Waiting、Succeeded、Failed	Claim、Executing、AwaitingResult 作为 Event 和 Attempt 阶段
Agent Session	Created、Running、Paused、Completed、Failed	Starting、WaitingInput、Cancelling、Recovering 作为运行阶段
Human Gate	Open、AwaitingDecision、Approved、ChangesRequested、Rejected、Expired	Presented、Blocked、Continue 等作为决定原因和事件
System Agent Intervention	Observed、Evaluated、Proposed、Applied、Closed	PolicyChecked、Measured、Promoted、Demoted、RolledBack 作为治理事件

原有第 23 节详细状态机仍然有效，但它属于 L2 运行诊断视图。任何客户端、跨端同步协议和第一版数据库主表只使用上表的主状态；不得把每个内部阶段都升级成新的公共状态。

44.2 核心字段预算

第一版核心对象只保留能驱动状态迁移、权限判断、关联追踪和恢复的字段。其余内容放在版本化 metadata、payload 或独立 Artifact 中，不把临时字段写入核心表。

对象	8-10 个核心字段	约束
Task	id, workspace_id, title, goal, state, graph_version, owner_id, risk, revision, created_at	参与者、标签和展示信息进入扩展对象
Task Node	id, task_id, kind, goal, state, depends_on, assignee_id, attempt, revision, updated_at	工具偏好和计划解释进入 Node metadata
Session	id, task_id, node_id, agent_id, state, context_ref, checkpoint_ref, revision, started_at, ended_at	Turn 和 ToolCall 保存在运行记录中
Assignment	id, node_id, actor_id, capability, lease_until, state, attempt, handoff_ref, revision	接管原因和策略决定进入 Event
Human Gate	id, task_id, node_id, kind, state, risk, expires_at, decision_by, decision_ref, revision	展示内容用 Evidence 引用，不复制全文

核心字段必须保持稳定；新增字段先放入扩展对象并通过 ADR
评估是否值得提升为核心字段。字段预算不是压缩事实，而是把事实放到正确的边界。

44.3 子模块合并规则

第一版按状态拥有权合并相似职责，避免为了名词完整而产生过多服务：

原有职责	第一版合并模块	合并依据
Task Model、Task Graph、Planner	Task Domain	都由 Control Plane 拥有 Task/Graph 版本和推进条件
Assignment、Handoff、Human Gate	Coordination	都决定“谁在什么条件下继续推进”
Session、Agent Loop、Context、JEV	Runtime Core	都属于一次可恢复 Agent 执行上下文
Tool Registry、Provider Gateway、Job Executor	Capability Execution	都把能力请求变成受控执行和结果
Event、Activity、Audit、Projection	Facts & Views	都从事实追加生成查询和审计视图
Memory、Evidence、Artifact	Evidence & Memory	都负责来源、版本和派生关系，但原始事实仍不可覆盖

合并不取消端口。模块内部可以继续有类、函数和目录，但跨模块只暴露六类稳定
Port：TaskPort、CoordinationPort、RuntimePort、ExecutionPort、FactsPort、EvidencePort。

45. 目标代码结构与依赖方向

以下是第一版目标仓库的结构图。它是实现蓝图，不声称当前工作区已经存在这些目录；每个包都必须在实现 PR 中映射到第 16 节的参考工程和第 42 节的证据等级。

```
platform/
apps/
  api/           # 外部 Command/Query、鉴权、版本协商
  worker-events/ # 投影、通知、Activity
  worker-governance/ # System Agent 观察、评估、建议
modules/
  task-domain/    # Task、Graph、Node、状态迁移
  coordination/   # Assignment、Lease、Handoff、Human Gate
  runtime-core/   # Session、Loop、Context、JEV Port
  capability-execution/# Tool、Provider、Policy、Job
  facts-views/    # Event、Activity、Audit、Projection
  evidence-memory/ # Artifact、Evidence、Memory、Provenance
  contracts/      # ID、Envelope、Error、Event schema
adapters/
  providers/      # CLI、API、SDK 适配器
  persistence/    # SQL、Event Store、Artifact Store
  transport/      # HTTP、WebSocket、Mobile Sync
  execution/      # Workspace、Sandbox、Remote Executor
tests/
  contract/       # Port 和 Event 契约测试
  integration/    # Task 端到端和跨端测试
  replay/         # Event/Artifact 回放
  fault-injection/ # 崩溃、超时、重复、只读和断线
```

依赖方向固定为 apps -> modules -> contracts，modules -> ports，adapters -> ports。领域模块不能反向依赖 HTTP、数据库 SDK 或具体 Provider；适配器只能实现端口，不能改变领域状态机。Rust 可以在 adapters/execution 或独立 Execution Host 中引入，但不改变 TypeScript 控制平面的领域契约。

46. 关键逻辑伪代码

伪代码只表达控制边界和不变量，不替代 L2

的具体类型。它使用与参考工程相同的“显式事件、显式结果、失败不吞掉”的风格。

46.1 Control Plane : 命令到事实

```
handle(command):
    actor = auth.require(command.actor_id)
    aggregate = task_repo.load(command.task_id)
    check_idempotency(command.request_id)
    policy.require(actor, command, aggregate)

    decision = task_domain.decide(aggregate, command)
    # decision contains next_state, events, and optional runtime_intent
    task_repo.save_state_and_events(decision.next_state, decision.events)

    if decision.runtime_intent:
        outbox.append(decision.runtime_intent)
    return accepted(aggregate.id, decision.next_state, aggregate.revision)
```

不变量：状态和领域事件在同一事务中提交；Runtime 只接收 Intent；重复的 request_id 不产生第二次副作用；客户端拿到的是受理结果，不是假装任务已经完成。

46.2 Agent Runtime : 循环、工具和停止

```
run(session, input):
    ctx = session.restore_or_start()
    pending = inbox.drain(session.id)

    while not stop_requested(ctx):
        message = model.stream(ctx.with(pending))
        emit(TurnStarted(session, message))
        calls = extract_tool_calls(message)

        if calls.empty:
            emit(TurnFinished(session, message))
            break

        results = []
        for call in choose_order(calls, ctx.tool_policy):
            prepared = tools.prepare(call, ctx)
            result = prepared.immediate or execution.run(prepared)
            results.append(result)
            emit(ToolFinished(session, call, result))

        ctx = ctx.append(message, results)
        checkpoint.save(session, ctx)
        pending = inbox.drain(session.id)

    return session.finish(ctx)
```

该结构直接对应 Pi 的 runLoop、工具调用分支和 AgentEventSink；JEV 作为 choose_order、局部规划和异步上下文维护的实现，可替换但不能改变 Session、ToolResult 和停止语义。

46.3 Provider : 策略、执行和证据

```
execute(intent):
    capability = provider_registry.resolve(intent.provider_id)
    policy.require(intent.actor, capability, intent.args)
    job = jobs.start(idempotency_key=intent.request_id)

    result = executor.run(capability, intent.args,
                          timeout=intent.timeout,
                          cancellation=intent.cancel_token)
    artifact_refs = artifacts.capture(result.outputs)
    evidence = evidence.record(job, result, artifact_refs)
    jobs.finish(job, result.status, evidence.id)
    return ProviderReport(job.id, result.status, artifact_refs, evidence.id)
```

Provider 不能直接改 Task 或 Node ; 它只返回 Job、Artifact 和 Evidence。Control Plane 根据报告决定是否推进 , 这个边界是避免“工具成功等于任务成功”的关键不变量。

47. 开源源码定位与引用清单

源码引用采用“仓库 / 相对路径 / 符号或行号 / 可复用模式 / 适用边界”的格式。行号以 v2.2 核对时的工作区源码为准；代码更新后需要重新核对。

工程	明确出处	可复用模式	本平台如何使用
pi-mono	pi-mono/packages/agent/src/agent-loop.ts:155-235 , 411-425 , 767-795	外层循环、工具调用顺序、事件 Sink、结果归一化	Runtime Core 的 Loop、ToolResult 和 Event 顺序
pi-mono	pi-mono/packages/agent/src/harness/session/session.ts:102-180	Session 对存储端口的封装、可查询记录和分支视图	Session Port、Checkpoint 和恢复读取
pi-mono	pi-mono/packages/agent/src/harness/compaction/compaction.ts:29-100	压缩结果、保留尾部、文件操作来源	Memory/Context 的异步整理和来源保留
deepseek-harness	deepseek-harness/packages/core/session/src/index.ts:2-38	Event-sourced Session、类型化事件和序列	Facts & Views 的事件追加和回放模式
deepseek-harness	deepseek-harness/packages/core/agent-loop/src/agent.ts:180-215	Driver phase、唤醒、取消和恢复边界	Runtime Session 的生命周期和停止语义
multica	multica/server/internal/daemon /、internal/events/、internal/realtime/	Daemon、事件通知、实时同步和设备连接	Communication 与 Worker 的跨端同步模式
ai-native	ai-native/backend/domain/services/parallel_execution_engine.py:91-169	DAG 层级、依赖检查和并行阶段执行	Task Graph 的规划与执行参考；不是通用 Control Plane
ai-native	ai-native/backend/service_layer/react_executor.py:33-196	Stage 循环、执行观察、Gate 记录和结果累积	Provider/Stage 执行的局部模式；仍需接入统一 Job/Evidence
codex	codex/codex-rs/app-server/、execpolicy/、thread-store/、rollout-trace/	App Server、执行策略、线程存储和 Trace	安全边界、审计和可恢复运行的工程参考

引用等级仍遵循 A/B/C：A 是可运行直接实现，B 是局部模式，C 是架构启发。所有引用都必须说明“复用什么”和“没有复用什么”，不能只贴目录名称。

48. ADR：第一版关键取舍

ADR-011：为什么第一版选 TypeScript，而不是 Rust

状态：已接受，适用于 Control Plane、Communication、Runtime Core 的第一版实现。

背景：参考工程中的 Pi 和 DeepSeek Harness 已经提供了 TypeScript 的 Agent Loop、Session、Tool、事件和测试模式；客户端、API 契约和共享类型也需要同一套快速演进的语言边界。Codex 的 Rust 代码对沙盒、执行策略和高可靠服务很有参考价值，但直接用 Rust 重写整个控制平面会同时引入新的异步模型、序列化边界和客户端类型生成成本。

决策：第一版领域模型、Control Plane、Runtime Core 和 API 使用 TypeScript；对沙盒、远程执行、资源隔离等高保证边界，允许在独立 Execution Host 或 Adapter 中使用 Rust。两种语言只通过版本化 Port、Envelope 和 Job 协议通信。

取舍：获得更快的产品迭代和对 Pi/DeepSeek 模式的直接复用，牺牲部分运行时性能和内存安全保证。高风险执行不靠语言选择单独解决，而靠沙盒、最小权限、Policy 和审计组合解决。

回滚/重评估条件：当 Runtime Host 的 CPU、内存、延迟或沙盒隔离指标连续两个版本达不到目标时，优先把热点 Adapter 或 Execution Host 迁移到 Rust，不迁移 Task 领域契约。

ADR-012：为什么第一版使用事件表，而不是直接使用事件流平台

状态：已接受，适用于第一版事实存储和投影。

背景：Task 状态、领域事件、Audit 和客户端查询需要事务一致的关联；第一版团队规模和任务吞吐不足以证明 Kafka 等独立事件流平台的运维成本。事件流和事件表解决的问题不同：事件表提供可查询、可回放的持久事实；事件流提供跨进程传输和消费扩展。

决策：使用关系数据库中的追加事件表作为事实来源，使用 Outbox/Worker 作为异步传输和投影机制。EventStore 通过 Port 抽象，未来可以把高吞吐通道接到事件流平台，但不改变 EventEnvelope、幂等和回放语义。

取舍：第一版获得简单事务、查询和备份，牺牲极端吞吐和跨地域流式扩展。任何外部消息队列都不能成为唯一事实来源；丢失传输消息必须能从事件表重建。

回滚/重评估条件：当事件追加、投影延迟、存储增长或跨域订阅达到 ADR 预设阈值时，引入流平台作为传输层，保留事件表作为可回放事实或明确迁移方案。

ADR-013：为什么使用 Career/Sequence，而不是简单 Level

状态：已接受，适用于 System Agent Governance。

背景：System Agent 的能力评价包含稳定的职业职责、逐步提升的序列条件、具体实例表现和负反馈约束。单一 Level 只能表达排序，不能表达“它被允许承担什么职责”“它通过了哪些证据”“它在哪些任务上失败”以及“暗面能力如何作为风险校验”。

决策：Career 表达角色职责和能力边界；Sequence 表达经过验证的阶段成长路线；Agent Instance 表达某个具体实例的当前 Profile、Evidence 和 Policy。对外可以派生一个展示用 level，但 level 不是权限来源，也不是升级依据。

取舍：模型多一个维度，评估和存档更复杂；换来可解释的升级条件、按职业分化的能力约束和可回滚的治理。升级必须由任务表现、Evidence、评估策略和人工/系统 Gate 共同决定，Agent 不能自评升级。

回滚/重评估条件：如果第一版只有单一职业和线性成长路线，可以暂时只启用一个 Career 和一条 Sequence；一旦出现角色分化、负反馈校验或跨任务能力迁移需求，不把它压平为不可逆的 Level 字段。

49. 版本化设计文档版式规范

从 v2.2 起，架构文档固定采用“封面 -> 文档状态 -> 目录 -> 目标与边界 -> 共享语言 -> L0 -> L1 -> 运行时与数据 -> 安全与质量 -> 实现证据 -> ADR ->

路线与检查表”的顺序。每章只在二级标题处换页；三级标题、表格和伪代码自然流动，避免为短小子节制造空白页。

每张架构图必须有编号、中文图题、所属层级和对应章节；每段伪代码必须标明它表达的边界和不变量；每条源码引用必须有仓库、相对路径、符号或行号和证据等级。封面只放标题、版本、日期和读者，不放解释性正文；目录只列一级章节，细节由章节内标题导航。

这套版式服务于评审：读者先理解决策，再查看结构和证据，最后检查实现路线。它不把视觉装饰当作架构质量，也不以页数代替细化程度。